

КЫРГЫЗ РЕСПУБЛИКАСЫНЫН БИЛИМ БЕРҮҮ ЖАНА ИЛИМ МИНИСТРЛИГИ  
МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ КЫРГЫЗСКОЙ РЕСПУБЛИКИ

И.Раззаков атындагы КЫРГЫЗ МАМЛЕКЕТТИК ТЕХНИКАЛЫК УНИВЕРСИТЕТИ  
КЫРГЫЗСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ имени  
И.Раззакова

КЕСИПТИК ОРТО БИЛИМ БЕРҮҮ (КОЛЛЕДЖ)  
СРЕДНЕЕ ПРОФЕССИОНАЛЬНОЕ ОБРАЗОВАНИЕ (КОЛЛЕДЖ)

## Выпускная квалификационная работа

на тему: «Проектирование базы данных для подсистемы расчетов с  
клиентами в телекоммуникационной компании»

Студент (ка) группы \_\_\_\_\_  
(№ учебной группы) (фамилия имя отчество полностью)(подпись)

Образовательная  
программа

230111 - Программирование в компьютерных системах  
(индекс и наименование специальности)

Форма обучения очная

Руководитель

\_\_\_\_\_  
(подпись)

\_\_\_\_\_  
(И.О. Фамилия)

Консультант  
(при наличии)

\_\_\_\_\_  
(подпись)

\_\_\_\_\_  
(И.О. Фамилия)

Председатель ПЦК

\_\_\_\_\_  
(подпись)

\_\_\_\_\_  
(И.О. Фамилия)

Бишкек – 2023

КЫРГЫЗ РЕСПУБЛИКАСЫНЫН БИЛИМ БЕРҮҮ ЖАНА ИЛИМ МИНИСТРЛИГИ  
МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ КЫРГЫЗСКОЙ РЕСПУБЛИКИ

И.Раззаков атындагы КЫРГЫЗ МАМЛЕКЕТТИК ТЕХНИКАЛЫК УНИВЕРСИТЕТИ  
КЫРГЫЗСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ имени  
И.Раззакова

КЕСИПТИК ОРТО БИЛИМ БЕРҮҮ (КОЛЛЕДЖ)  
СРЕДНЕЕ ПРОФЕССИОНАЛЬНОЕ ОБРАЗОВАНИЕ (КОЛЛЕДЖ)

**“БЕКТЕМИН”**

**“УТВЕРЖДАЮ”**

«Башкаруу жана IT» бөлүмүнүн башчысы  
Заведующая отд. «Управления и IT»

Б.Т.Ткачева

“\_\_” \_\_\_\_ 2023-ж.г.

Бүтүрүүчү квалификациялык ишине (БКИ)

**ТАПШЫРМА**

**ЗАДАНИЕ**

на выпускную квалификационную работу (ВКР)

Топтун студенти/Студент группы Асылбеков Айдарбек Аскарбекович

(фамилиясы, аты, атасынын аты / фамилия, имя, отчество)

1. БКИ аталышы \_\_\_\_\_

Тема ВКР \_\_\_\_\_

Буйрук менен бекиген \_\_\_\_\_ “\_\_” \_\_\_\_ 20\_\_-ж.г.

Утверждена приказом

2. Бүтүрүлгөн ишти студенттин берүү

мөөнөтү “\_\_” \_\_\_\_ 20\_\_-ж.г.

Срок сдачи студентом законченной работы

3. Ишти аткарууга алгачкы маалыматтар

**Исходные данные к работе**

---

---

---

---

---

---

| Катар №<br>№ п/п | Эсептеп-түшүндүрмө каттын мазмуну<br>(иштетүүгө тийиштүү маселелердин тизмеси)<br>Содержание расчетно-пояснительной записки<br>(перечень, подлежащих разработке вопросов) | Көлөмү,<br>% менен<br>Объем, в % | Аткаруу<br>мөөнөтү<br>Срок<br>выполнения |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------|------------------------------------------|
| 1.               |                                                                                                                                                                           |                                  |                                          |
| 2.               |                                                                                                                                                                           |                                  |                                          |
| 3.               |                                                                                                                                                                           |                                  |                                          |
| 4.               |                                                                                                                                                                           |                                  |                                          |
| 5.               |                                                                                                                                                                           |                                  |                                          |
| 6.               |                                                                                                                                                                           |                                  |                                          |
| 7.               | <i>Колдонулган адабияттар / Список литературы</i>                                                                                                                         |                                  |                                          |
| 8.               | <i>Бардыгы / Итого</i>                                                                                                                                                    | <i>100</i>                       |                                          |

| Катар №<br>№ п/п | Чийме бөлүгү<br>Графическая часть | 24 форм.<br>барагы<br>Лист 24-<br>форм. | Көлөмү,<br>% менен<br>Объем, в % | Аткаруу<br>мөөнөтү<br>Срок<br>выполн. |
|------------------|-----------------------------------|-----------------------------------------|----------------------------------|---------------------------------------|
| 1.               |                                   |                                         |                                  |                                       |
| 2.               |                                   |                                         |                                  |                                       |
| 3.               |                                   |                                         |                                  |                                       |
| 4.               |                                   |                                         |                                  |                                       |
| 5.               | <i>Бардыгы / Итого</i>            |                                         | <i>100</i>                       |                                       |

Айрым бөлүктөрү боюнча консультациялар / Консультации по отдельным разделам  
(жетекчисинен тышкары / помимо руководителя)

| Катар №<br>№ п/п | Бөлүктөрдүн аталыштары<br>Наименование разделов | Фамилиясы, аты, атасынын аты<br>Фамилия, имя, отчество | Колу<br>Подпись |
|------------------|-------------------------------------------------|--------------------------------------------------------|-----------------|
| 1.               | Изилдөө бөлүмү<br>Исследовательская часть       |                                                        |                 |
| 2.               | Практикалык бөлүм<br>Практическая часть         |                                                        |                 |
| 3.               | Көзөмөл нормасы<br>Нормоконтроль                |                                                        |                 |

Тапшырма берген

күнү

“\_\_” \_\_\_\_\_ 20\_\_-ж.г.

Дата выдачи задания

БКИ жетекчиси

Руководитель ВКР

\_\_\_\_\_

фамилиясы, аты, атасынын аты, окумуштуулук даражасы, наамы, колу / ФИО, уч.степень,  
звание, подпись

Тапшырманы алган

күнү

“\_\_” \_\_\_\_\_ 20\_\_-ж.г.

Дата получения

задания

\_\_\_\_\_

(студенттин аты-жөнү,  
колу / ФИО студента, подпись)

## **СОДЕРЖАНИЕ**

|                                                                     |    |
|---------------------------------------------------------------------|----|
| ВВЕДЕНИЕ .....                                                      | 5  |
| ГЛАВА 1. АНАЛИЗ ПРОЕКТИРОВАНИЯ БАЗЫ ДАННЫХ.....                     | 7  |
| 1.1. Возможности проектирование базы данных .....                   | 8  |
| 1.2. Совершенствование обеспечения проектирования базы данных ...   | 12 |
| 1.3. Политика проектирования базы данных .....                      | 13 |
| ГЛАВА 2 РЕАЛИЗАЦИЯ ПРОГРАММ ДЛЯ ПРОЕКТИРОВАНИЕ<br>БАЗЫ ДАННЫХ ..... | 20 |
| 2.1 ФОРМУЛИРОВКА СУЩНОСТЕЙ .....                                    | 22 |
| 2.2 Создание базы данных .....                                      | 23 |
| 2.3 Создание таблиц .....                                           | 25 |
| 2.4 Заполнение таблиц .....                                         | 31 |
| 2.5 Создание запросов .....                                         | 34 |
| 2.6 Создание отчетов .....                                          | 40 |
| ЗАКЛЮЧЕНИЕ.....                                                     | 44 |
| СПИСОК ИСПОЛЬЗОВАННЫХ ЛИТЕРАТУР .....                               | 46 |

## **ВВЕДЕНИЕ**

В наши дни, когда телекоммуникационные компании играют ключевую роль в современном обществе, важно иметь эффективную систему учета и расчетов с клиентами. Они обслуживают огромное количество клиентов и предоставляют широкий спектр услуг, таких как мобильная связь, интернет-подключение, телевидение и другие. Проектирование базы данных для подсистемы расчетов – важный этап, который позволяет обеспечить надежность, эффективность и безопасность всех процессов внутри компании. В связи с ростом конкуренции и сложности бизнес-процессов, эффективное управление клиентской информацией и точные расчеты с клиентами становятся важными задачами для телекоммуникационных компаний.

Для успешного функционирования подсистемы расчетов с клиентами, требуется грамотное проектирование базы данных, которая будет хранить, обрабатывать и обеспечивать доступность всех необходимых данных. Проектирование базы данных является фундаментальной частью разработки информационных систем и позволяет оптимизировать работу компании, повысить эффективность бизнес-процессов и обеспечить качественное обслуживание клиентов.

Подсистема расчетов с клиентами в телекоммуникационной компании включает в себя такие процессы, как формирование и учет счетов, определение тарифов и условий платежей, контроль за расчетами и взаиморасчетами с клиентами. Вся эта информация должна быть аккуратно организована и доступна для использования в режиме реального времени. Эффективное проектирование базы данных позволяет управлять сложностью данных, обеспечивать целостность и безопасность информации, а также повышать производительность системы.

Целью данной работы является рассмотрение основных принципов и подходов к проектированию базы данных для подсистемы расчетов с клиентами в телекоммуникационной компании. Мы изучим различные модели

данных, методы нормализации, выбор структуры таблиц, оптимизацию запросов и другие аспекты, необходимые для разработки эффективной базы данных. Благодаря грамотному проектированию базы данных, телекоммуникационные компании смогут улучшить свою операционную эффективность, повысить уровень обслуживания клиентов и достичь конкурентных преимуществ на рынке.

## ГЛАВА 1 АНАЛИЗ ПРОЕКТИРОВАНИЯ БАЗЫ ДАННЫХ

Проектирование базы данных для подсистемы расчетов с клиентами в телекоммуникационной компании является критическим шагом для обеспечения эффективного управления клиентской информацией и точных расчетов. В этом разделе мы рассмотрим основные аспекты, которые следует учитывать при проектировании такой базы данных.

**Архитектура базы данных:** Первый шаг в проектировании базы данных - определение ее архитектуры. В случае подсистемы расчетов с клиентами в телекоммуникационной компании, может быть использована клиент-серверная архитектура, где клиенты (например, биллинговая система или веб-портал) взаимодействуют с сервером базы данных для получения и обновления информации.

**Модель данных:** Следующий шаг - выбор подходящей модели данных. В данном случае, реляционная модель данных является широко применяемым вариантом, так как она обеспечивает структурированное хранение данных в таблицах с использованием ключей и отношений. Реляционная модель позволяет легко определить сущности, атрибуты и связи между ними.

**Нормализация данных:** Нормализация данных является важной частью проектирования базы данных. Она позволяет устранить избыточность данных и обеспечить целостность информации. Применение нормализации до нужного уровня (например, третья нормальная форма или выше) помогает избежать проблем с обновлением, вставкой и удалением данных.

**Определение сущностей и связей:** Для подсистемы расчетов с клиентами необходимо определить ключевые сущности, такие как клиенты, тарифы, счета, платежи и договоры. Затем следует определить связи между этими сущностями, такие как связь "один-ко-многим" между клиентами и счетами, а также связи между клиентами и тарифами.

**Индексы и оптимизация запросов:** Создание подходящих индексов на таблицах базы данных помогает ускорить выполнение запросов и повысить

производительность системы. Анализ типичных запросов, которые будут выполняться в подсистеме расчетов с клиентами, помогает определить наиболее эффективные индексы.

**Безопасность данных:** При проектировании базы данных необходимо учитывать вопросы безопасности данных. Важно определить соответствующие уровни доступа и защиту данных, чтобы обеспечить конфиденциальность и целостность информации о клиентах и расчетах.

В целом, проектирование базы данных для подсистемы расчетов с клиентами в телекоммуникационной компании требует тщательного анализа требований, моделирования данных и учета особенностей бизнес-процессов компании. Корректное проектирование базы данных позволяет оптимизировать расчеты с клиентами, улучшить операционную эффективность и обеспечить качественное обслуживание клиентов.

### **1.1. Возможности проектирование базы данных**

**Централизованное хранение данных:** Проектирование базы данных позволяет создать единую централизованную систему хранения клиентской информации, такой как данные о клиентах, тарифах, счетах, платежах и договорах. Это обеспечивает доступность и консолидацию данных для всех подсистем и процессов, связанных с расчетами с клиентами.

**Улучшенная точность расчетов:** Проектирование базы данных позволяет внедрить сложные расчетные алгоритмы и правила, которые обеспечивают точность и надежность процесса расчетов с клиентами. База данных может содержать информацию о тарифных планах, скидках, условиях оплаты и других факторах, необходимых для правильного расчета сумм для клиентов.

**Гибкость и масштабируемость:** Проектирование базы данных позволяет создать гибкую и масштабируемую систему, которая может адаптироваться к изменяющимся требованиям бизнеса и растущему объему данных. Это



позволяет компании легко добавлять новые функции и услуги, а также обрабатывать все больше клиентов без значительных изменений в архитектуре базы данных.

Улучшенная аналитика и отчетность: Проектирование базы данных для подсистемы расчетов с клиентами позволяет хранить и анализировать большие объемы данных о клиентах и их расчетах. Это дает возможность компании проводить различные аналитические и статистические исследования, а также создавать подробные отчеты о расчетах, доходности, анализе клиентского поведения и других аспектах бизнеса.

Интеграция с другими системами: Проектирование базы данных позволяет интегрировать подсистему расчетов с клиентами с другими информационными системами компании, такими как CRM (управление взаимоотношениями с клиентами), ERP (система планирования ресурсов предприятия) и др. Это позволяет обмениваться данными, автоматизировать процессы и улучшить взаимодействие между различными бизнес-подразделениями. При проектировании БД могут появиться нежелательные свойства, такие как избыточность, аномалии обновления, аномалии включения, аномалии удаления и др. Для уменьшения нежелательных характеристик БД к схемам отношений применяют процедуры нормализации.

Нормализация - это разбиение таблицы на две или более, обладающие лучшими свойствами при включении, изменении и удалении данных. Окончательная цель нормализации сводится к получению такого проекта базы данных, в котором каждый факт появляется лишь в одном месте, т.е. исключена избыточность информации. Это делается не столько с целью экономии памяти, сколько для исключения возможной противоречивости хранимых данных.

Нормализация – это процесс разделения информации на структурные единицы, т.е. таблицы.

Нормализация БД должна быть выполнена с учётом следующего

правила: таблицы, которые содержат повторяющуюся информацию, для устранения дублирования значений должны быть разделены на отдельные таблицы, что приводит к сокращению размеров БД.

В теории реляционных баз данных вводятся понятия так называемых “нормальных форм” — требований к организации данных в таблицах.

Нормальные формы нумеруются последовательно, по мере ужесточения требований. В правильно спроектированной БД таблицы находятся как минимум в третьей нормальной форме.

Так же можно сказать, что процесс нормализации представляет собой приведение таблиц к требуемому уровню нормальности: первый, второй и третий. Каждый уровень нормальности соответствует определённой нормальной форме.

Теория нормализации основана на концепции нормальных форм. Говорят, что таблица находится в данной нормальной форме, если она удовлетворяет определённому набору требований. Теоретически существует пять нормальных форм, но на практике обычно используются только первые три. Первые две нормальные формы являются промежуточными шагами для приведения базы данных к третьей нормальной форме.

#### Первая нормальная форма (1НФ)

Первая нормальная форма предписывает, что все данные, содержащиеся в таблице, должны быть атомарными (неделимыми). Перечень соответствующих атомарных типов данных определяется СУБД. Требование 1НФ совершенно естественное. Оно означает, что в каждом поле каждой записи должна находиться только одна величина, но не массив и не какая-либо другая структура данных.

#### Вторая нормальная форма (2НФ)

Говорят, что таблица находится во второй нормальной форме, если она находится в 1НФ и каждый не ключевой столбец полностью зависит от первичного ключа. Другими словами, значение каждого поля должно

полностью определяться значением первичного ключа. Важно отметить, что зависимость от первичного ключа понимается именно как зависимость от ключа целиком, а не от отдельной его составляющей (в случае составного ключа).

Чтобы перейти от первой нормальной формы ко второй, нужно выполнить следующую последовательность действий:

Определить, на какие части можно разбить первичный ключ, так чтобы некоторые из неключевых полей зависели от одной из этих частей (эти части не обязаны состоять из одной колонки).

Создать новую таблицу для каждой такой части ключа и группы, зависящих от нее полей и переместить их в эту таблицу. Часть бывшего первичного ключа станет при этом первичным ключом новой таблицы.

Удалить из исходной таблицы поля, перемещенные в другие таблицы, кроме тех, которые станут внешними ключами.

Третья нормальная форма (3НФ)

Говорят, что таблица находится в 3НФ, если она соответствует 2НФ и все не ключевые столбцы взаимно независимы.

Взаимную зависимость столбцов удобно понимать следующим образом: столбцы являются взаимно зависимыми, если нельзя изменить один из них, не изменяя другой.

Чтобы перейти от второй нормальной формы к третьей, нужно выполнить следующую последовательность действий:

Определить все поля, от которых зависят другие поля.

Создать новую таблицу для каждого такого поля или группы полей и группы зависящих от него полей и переместить их в эту таблицу. Поле или группа полей, от которого зависят перемещенные поля, станет при этом первичным ключом новой таблицы.

Удалить из исходной таблицы поля, перемещенные в другие таблицы, кроме тех, которые станут внешними ключами.

## Высшие нормальные формы

В теории реляционных баз данных рассматриваются и формы высших порядков — нормальная форма Бойса — Кодда, 4НФ, 5НФ и даже выше. Большого практического значения эти формы не имеют, и разработчики, как правило, всегда останавливаются на 3НФ.

Проектирование базы данных для подсистемы расчетов с клиентами в телекоммуникационной компании предоставляет широкий спектр возможностей для оптимизации бизнес-процессов, улучшения точности расчетов, управления клиентской информацией и анализа данных. Это способствует эффективной работе компании и удовлетворению потребностей клиентов.

### **1.2. Совершенствование обеспечения проектирования базы данных**

Совершенствование обеспечения проектирования базы данных для подсистемы расчетов с клиентами в телекоммуникационной компании является важным шагом для оптимизации процессов и улучшения качества обслуживания клиентов. Вот некоторые возможности для совершенствования обеспечения проектирования базы данных:

**Автоматизация процессов:** Использование автоматизированных инструментов и средств для проектирования баз данных позволяет ускорить процесс разработки и уменьшить вероятность ошибок. Это может включать CASE-средства (Computer-Aided Software Engineering), которые помогают создавать модели данных, генерировать код и проверять целостность базы данных.

**Использование современных технологий и инструментов:** Проектирование базы данных для подсистемы расчетов с клиентами может быть улучшено с использованием современных технологий и инструментов. Например, использование облачных баз данных позволяет гибко масштабировать и хранить данные, а NoSQL-базы данных могут обеспечить

быстрый доступ и обработку больших объемов данных.

Управление производительностью: Оптимизация производительности базы данных является важным аспектом совершенствования ее обеспечения. Использование индексов, правильное проектирование запросов и оптимизация структуры таблиц позволяют снизить время выполнения запросов и улучшить общую производительность системы.

Обеспечение безопасности данных: в условиях угроз кибербезопасности, обеспечение безопасности данных становится все более важным. Совершенствование обеспечения базы данных включает в себя внедрение механизмов шифрования данных, контроля доступа, резервного копирования и мониторинга, чтобы защитить конфиденциальность и целостность информации о клиентах.

Непрерывное совершенствование: Проектирование базы данных является непрерывным процессом. Необходимо постоянно анализировать требования бизнеса и клиентов, а также проводить регулярное обследование и оптимизацию базы данных. Это позволяет адаптировать систему к изменяющимся условиям и эффективно управлять расчетами с клиентами.

Совершенствование обеспечения проектирования базы данных для подсистемы расчетов с клиентами в телекоммуникационной компании позволяет повысить эффективность, надежность и безопасность системы, обеспечивая лучшее обслуживание клиентов и достижение бизнес-целей компании.

### **1.3 Политика проектирования базы данных**

Разработка политики проектирования базы данных для подсистемы расчетов с клиентами в телекоммуникационной компании является важной задачей, которая помогает обеспечить консистентность, эффективность и безопасность базы данных. Вот некоторые ключевые аспекты, которые следует учесть при разработке такой политики:

Стандарты и принципы проектирования: определите стандарты и принципы проектирования базы данных, которые будут использоваться в телекоммуникационной компании. Это может включать выбор определенных моделей данных (например, реляционная модель), нормализацию данных до нужного уровня и определение структуры таблиц и связей.

Доступность и масштабируемость: установите требования к доступности базы данных и ее масштабируемости. Рассмотрите возможность использования репликации данных, кластеризации или облачных решений, чтобы обеспечить высокую доступность и возможность масштабирования системы при увеличении объема данных и нагрузки.

Нормализация данных: определите политику нормализации данных, которая будет применяться при проектировании базы данных. Установите правила для устранения избыточности данных и обеспечения целостности информации. Это помогает уменьшить дублирование данных и обеспечить последовательность и точность расчетов.

Индексирование и оптимизация запросов: определите политику по использованию индексов и оптимизации запросов. Установите правила для создания индексов на таблицах базы данных, основанных на типичных запросах, выполняемых подсистемой расчетов с клиентами. Это помогает ускорить выполнение запросов и повысить производительность системы.

Безопасность данных: Разработайте политику безопасности данных, которая обеспечит защиту конфиденциальности, целостности и доступности данных. Установите правила для контроля доступа к базе данных, шифрования данных, резервного копирования и мониторинга активности, чтобы предотвратить несанкционированный доступ и минимизировать риски утечки информации.

Обслуживание и сопровождение: установите политику обслуживания и сопровождения базы данных. Определите процедуры для резервного копирования и восстановления данных, мониторинга производительности,

регулярного обновления программного обеспечения и применения патчей. Это помогает обеспечить надежную работу системы и минимизировать возможные проблемы.

**Аудит и контроль:** установите политику аудита и контроля базы данных. Определите процедуры для регистрации изменений данных, отслеживания активности пользователей и реализации механизмов контроля целостности данных. Это позволяет обеспечить прозрачность и отслеживаемость операций с данными, а также выявлять и предотвращать возможные нарушения безопасности.

Важно помнить, что политика проектирования базы данных должна быть документирована и коммуницирована всем участникам, работающим с базой данных. Она должна быть гибкой и адаптируемой к изменениям в бизнес-требованиях и технологической среде. Постоянное обновление и улучшение политики является ключевым аспектом успешной разработки базы данных для подсистемы расчетов с клиентами в телекоммуникационной компании.

#### *1.4.1 Основные положения проектирование базы данных для подсистемы расчетов с клиентами в телекоммуникационной компании*

При проектировании базы данных для подсистемы расчетов с клиентами в телекоммуникационной компании следует учесть несколько основных положений. Вот некоторые из них:

**Централизованное хранение данных:** База данных должна обеспечивать централизованное хранение всех необходимых данных о клиентах, включая информацию о подключенных услугах, тарифах, платежах и других финансовых операциях. Централизованное хранение данных позволяет упростить управление и обработку информации о клиентах.

**Нормализация данных:** Применение нормализации данных помогает устранить избыточность и несогласованность данных. Разделение данных на

логически связанные таблицы и установление связей между ними обеспечивает целостность и согласованность информации в базе данных.

**Использование уникальных идентификаторов:** Каждому клиенту следует назначить уникальный идентификатор, который будет использоваться для идентификации и связи с другими таблицами базы данных. Это позволяет эффективно выполнять операции поиска, сортировки и связывания данных.

**Архитектура базы данных:** Разработка правильной архитектуры базы данных играет важную роль в эффективности и производительности системы. Рассмотрите использование реляционной модели данных и оптимальные структуры таблиц для хранения информации о клиентах и их расчетах. Также можно рассмотреть возможность использования индексов, представлений и хранимых процедур для ускорения запросов и обработки данных.

**Безопасность данных:** Защита данных клиентов является критически важной. Разработка соответствующих политик безопасности, включая ограничение доступа к базе данных, шифрование конфиденциальной информации и мониторинг активности пользователей, помогает предотвратить несанкционированный доступ и снизить риски утечки данных.

**Резервное копирование и восстановление данных:** Регулярное резервное копирование данных и разработка процедур восстановления помогают защитить информацию о клиентах от потери или повреждения. Важно установить правильную стратегию резервного копирования, которая соответствует бизнес-требованиям и минимизирует потенциальное воздействие на работу системы в случае сбоя.

**Масштабируемость:** Система должна быть способной масштабироваться для обработки растущего объема данных и увеличивающейся нагрузки. Рассмотрите возможность использования горизонтального и вертикального масштабирования, а также распределенных архитектур баз данных, чтобы обеспечить гибкость и эффективность работы системы.



Учитывая эти основные положения, можно разработать базу данных, которая эффективно поддерживает подсистему расчетов с клиентами в телекоммуникационной компании, обеспечивая надежность, производительность и безопасность данных. С точки зрения внешнего представления объектов реального мира модель данных — это основные понятия и способы, используемые при анализе и описании предметной области.

Среди многих попыток представить обработку данных на формальном абстрактном уровне реляционная модель, предложенная Э. Ф. Коддом, стала по существу первой работоспособной моделью данных, поскольку помимо средств описания объектов имела эффективный инструментарий преобразований этих описаний — операции реляционной алгебры.

Реляционная алгебра в том виде, в котором она была определена Э. Ф. Коддом, состоит из двух групп по четыре оператора.

1. Традиционные операции: объединение, пересечение, разность и декартово произведение.

2. Специальные реляционные операции: выборка, проекция, соединение, деление.

Объединение возвращает таблицу, содержащую все записи, которые принадлежат либо одной из двух заданных таблиц, либо им обоим.

Пересечение возвращает таблицу, содержащую все записи, которые принадлежат одновременно двум заданным таблицам.

Разность возвращает таблицу, содержащую все записи, которые принадлежат первому из двух заданных таблиц и не принадлежат второй.

Выборка возвращает таблицу, содержащую все записи из заданной таблицы, которые удовлетворяют указанным условиям.

Проекция возвращает таблицу, содержащую все записи заданной таблицы, которые остались в этой таблице после исключения из неё

некоторых атрибутов

Соединение возвращает таблицу, содержащую все возможные записи, которые представляют собой комбинацию атрибутов двух записей, принадлежащих двум заданным таблицам, при условии, что в этих двух комбинированных записях присутствуют одинаковые значения в одном или нескольких общих для исходных таблиц атрибутах (причем эти общие значения в результирующей записи появляются один раз, а не дважды).

Деление для заданных двух унарных таблиц и одной бинарной возвращает таблицу, содержащую все записи из первой унарной таблицы, которые содержатся также в бинарной таблице и соответствуют всем записям во второй унарной таблице.

Результат выполнения любой операции над таблицами также является таблицей, поэтому результат одной операции может использоваться в качестве исходных данных для другой. Другими словами, можно записывать вложенные реляционные выражения, т. е. выражения, в которых операторы сами представлены реляционными выражениями, причем произвольной сложности. Эта особенность называется свойством реляционной замкнутости.

#### *1.4.2 Организационные меры обеспечения проектирование базы данных для подсистемы расчетов с клиентами в телекоммуникационной компании*

Организационные меры играют важную роль в обеспечении успешного проектирования базы данных для подсистемы расчетов с клиентами в телекоммуникационной компании. Вот некоторые из них:

Установление команды проекта: Сформируйте команду проекта, которая будет отвечать за проектирование базы данных. В команду включите экспертов по базам данных, аналитиков, разработчиков и представителей бизнеса. Каждый член команды должен иметь ясное понимание целей проекта и своих ролей и обязанностей.

Определение бизнес-требований: тщательно определите бизнес-требования, которые должна удовлетворять база данных подсистемы

расчетов. Это включает определение необходимых данных о клиентах, требований к производительности, безопасности и интеграции с другими системами в компании. Четкое понимание бизнес-требований поможет сосредоточиться на правильном проектировании базы данных.

Участие заинтересованных сторон: включите заинтересованные стороны, такие как менеджеры отделов продаж, финансов, маркетинга и клиентского обслуживания, в процесс проектирования базы данных. Они могут предоставить ценные знания о требованиях клиентов, типичных расчетах и операционных процессах, которые должны быть учтены при проектировании базы данных.

Документирование и коммуникация: важно документировать все аспекты проектирования базы данных и регулярно обмениваться информацией с членами команды и заинтересованными сторонами. Создание детальных спецификаций, диаграмм базы данных и других документов поможет установить ясное понимание требований и обеспечить согласованность в процессе разработки.

Процессы контроля качества: внедрите процессы контроля качества, которые позволяют проверять и проверять базу данных на соответствие установленным требованиям и стандартам. Это может включать проведение регулярных проверок кода, тестирование производительности, аудит базы данных и обратную связь от пользователей.

Обучение и поддержка: Обеспечьте обучение и поддержку для команды проекта и пользователей, которые будут работать с базой данных. Это поможет им развить необходимые навыки и знания для эффективного использования базы данных и минимизации возможных ошибок.

Применение этих организационных мер поможет обеспечить более эффективное и качественное проектирование базы данных для подсистемы расчетов с клиентами в телекоммуникационной компании.

## **ГЛАВА 2 РЕАЛИЗАЦИЯ ПРОГРАММ ДЛЯ ПРОЕКТИРОВАНИЯ БАЗЫ ДАННЫХ ДЛЯ ПОДСИСТЕМЫ РАСЧЕТОВ С КЛИЕНТАМИ В ТЕЛЕКОММУНИКАЦИОННОЙ КОМПАНИИ**

В данной главе рассматривается реализация программ для проектирования базы данных для подсистемы расчетов с клиентами в телекоммуникационной компании. Этот этап является важным и предполагает создание физической структуры базы данных на основе разработанных ранее концептуальных и логических моделей.

### **Выбор СУБД**

Первым шагом в реализации программ для проектирования базы данных является выбор подходящей системы управления базами данных (СУБД). При выборе СУБД необходимо учитывать требования проекта, такие как производительность, масштабируемость, безопасность, совместимость с другими системами, доступность функций и техническая поддержка. Некоторые популярные СУБД, которые широко используются в телекоммуникационных компаниях, включают Oracle, MySQL, Microsoft SQL Server и PostgreSQL.

### **Физическое проектирование**

Физическое проектирование базы данных включает создание таблиц, определение полей и их типов данных, установление ограничений целостности, индексирование и оптимизацию структуры базы данных для обеспечения эффективного хранения и доступа к данным. На этом этапе разрабатываются физические модели данных, которые представляют собой набор таблиц и связей между ними, а также определяются атрибуты и их характеристики.

### **Создание таблиц и связей**

На основе физической модели данных создаются таблицы в выбранной СУБД. Каждая таблица представляет собой сущность или отношение из логической модели данных. Затем определяются связи между таблицами, такие как отношения один-к-одному, один-ко-многим и многие-ко-многим.

Связи обеспечивают целостность данных и позволяют эффективно извлекать информацию из базы данных.

#### Определение полей и их типов данных

Для каждой таблицы определяются поля и их типы данных. Типы данных должны соответствовать хранящейся информации и требованиям приложений. Например, для хранения имени клиента может использоваться тип данных VARCHAR, а для хранения суммы платежа - тип данных DECIMAL. Также определяются ограничения полей, такие как обязательность, уникальность и значения по умолчанию.

#### Ограничения целостности

Важным аспектом физического проектирования базы данных является установление ограничений целостности. Ограничения целостности определяют правила, которым должны соответствовать данные в базе данных. Например, можно установить ограничение на то, что поле "Дата активации услуги" не может быть пустым или ограничение на уникальность номера телефона клиента. Ограничения целостности помогают поддерживать согласованность и точность данных.

#### Индексирование и оптимизация

Для обеспечения быстрого доступа к данным и повышения производительности базы данных следует использовать индексы. Индексы создаются для определенных полей и ускоряют поиск, сортировку и объединение данных. При создании индексов следует учитывать типы запросов, которые часто выполняются в подсистеме расчетов с клиентами. Кроме того, проводится оптимизация структуры базы данных, такая как размещение таблиц на разных дисковых устройствах для распределения нагрузки и улучшения производительности.

#### Тестирование и отладка

После реализации программ для проектирования базы данных следует провести тестирование и отладку системы. Это включает проверку

корректности данных, работоспособности запросов и функциональности приложений, использующих базу данных. При обнаружении ошибок или несоответствий требованиям производится исправление и повторное тестирование.

В данной главе представлены этапы и процессы, связанные с реализацией программ для проектирования базы данных для подсистемы расчетов с клиентами в телекоммуникационной компании. Корректная реализация базы данных играет важную роль в обеспечении эффективного функционирования подсистемы расчетов и обработке данных клиентов.

## **2.1 ФОРМУЛИРОВКА СУЩНОСТЕЙ**

*При обследовании предметной области* были выделены следующие сущности:

- «Клиенты» (Customers)
- «Тарифы» (Tariffs)
- «Счета» (Invoices)

Сущность «Клиенты» содержит информацию о клиенте по следующим параметрам:

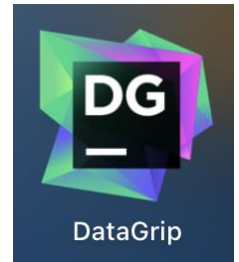
- ID (уникальный идентификатор клиента),
- Имени (name),
- Адрес электронной почты (email),
- Номер телефона (phone),
- Адрес проживания (address),
- Дата регистрации (registration\_date)

Сущность «Тарифы» содержит информацию о клиенте по следующим параметрам:

- ID (уникальный идентификатор тарифа),

- Название (name),
- Стоимость (price),
- Описание (description)

Сущность «Счета» содержит информацию о клиенте по следующим параметрам:



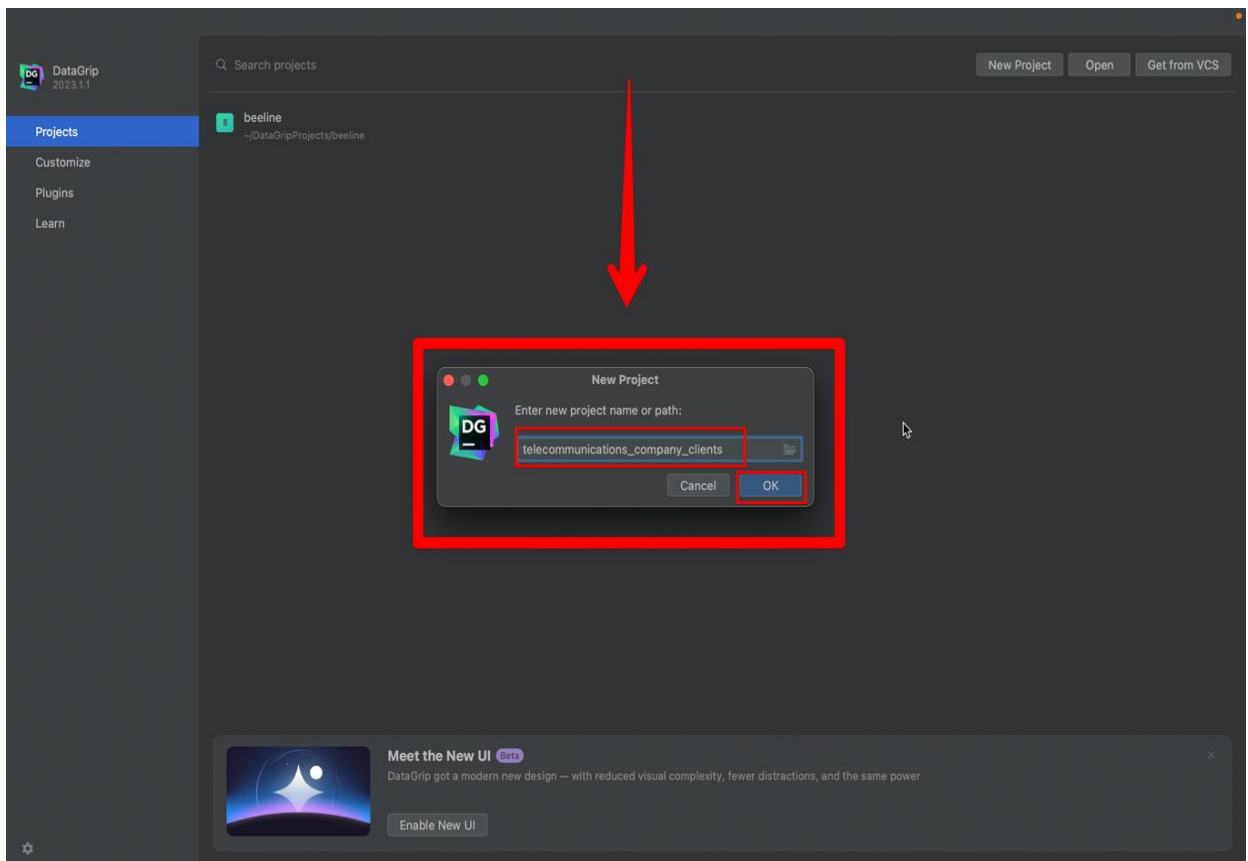
- ID (уникальный идентификатор счета)
- ID клиента (customer\_id)
- Дата выставления (issue\_date)
- Дата оплаты (payment\_date)
- Сумма (amount)
- Статус (status)



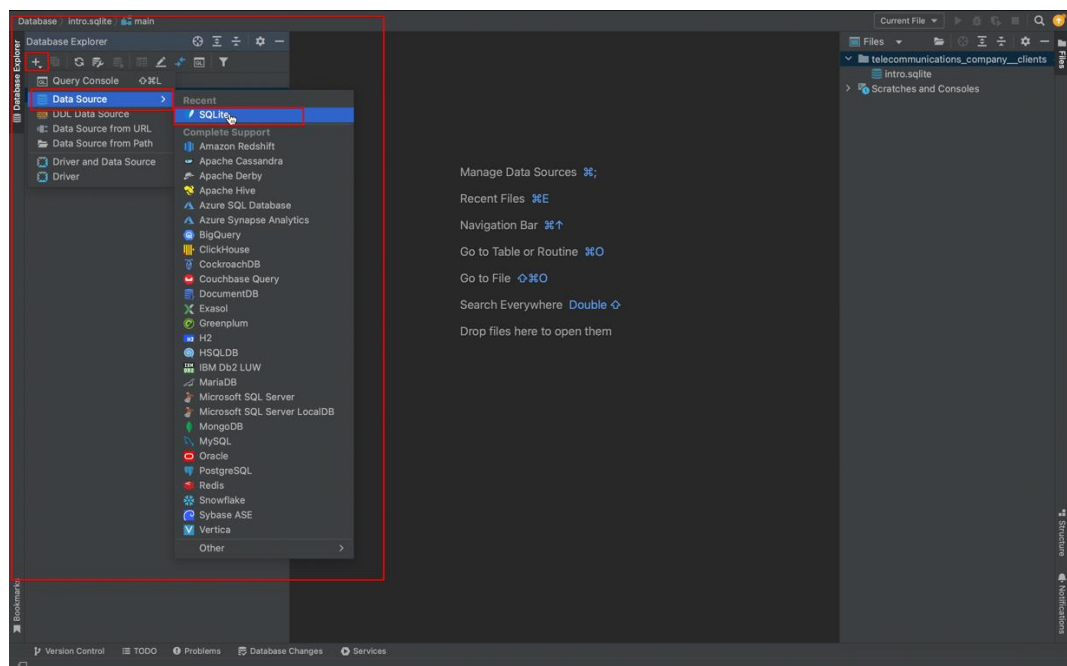
## 2.2 Создание базы данных

Для создания базы данных, я буду использовать интегрированную среду разработки (IDE) DataGrip от компании JetBrains.

А в качестве СУБД я буду использовать SQLite. База данных — это набор структурированной информации. Для ее изменения требуются системы управления — СУБД. Как и любая СУБД, SQLite позволяет записывать новую и запрашивать существующую информацию, изменять ее, настраивать доступ. Открываем интегрированную среду разработки (IDE) DataGrip, и создаем новый проект, проект будет называться “telecommunications\_company\_clients”.

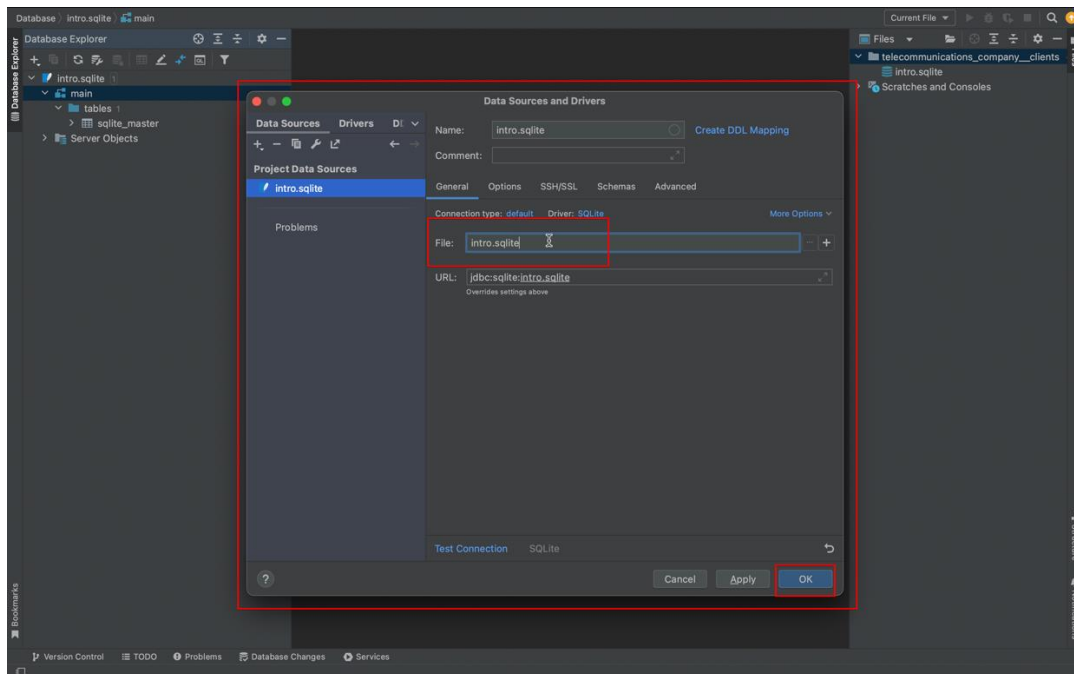


После создания проекта выбираем СУБД в левом меню самой программы, в нашем случае это “SQLite”.



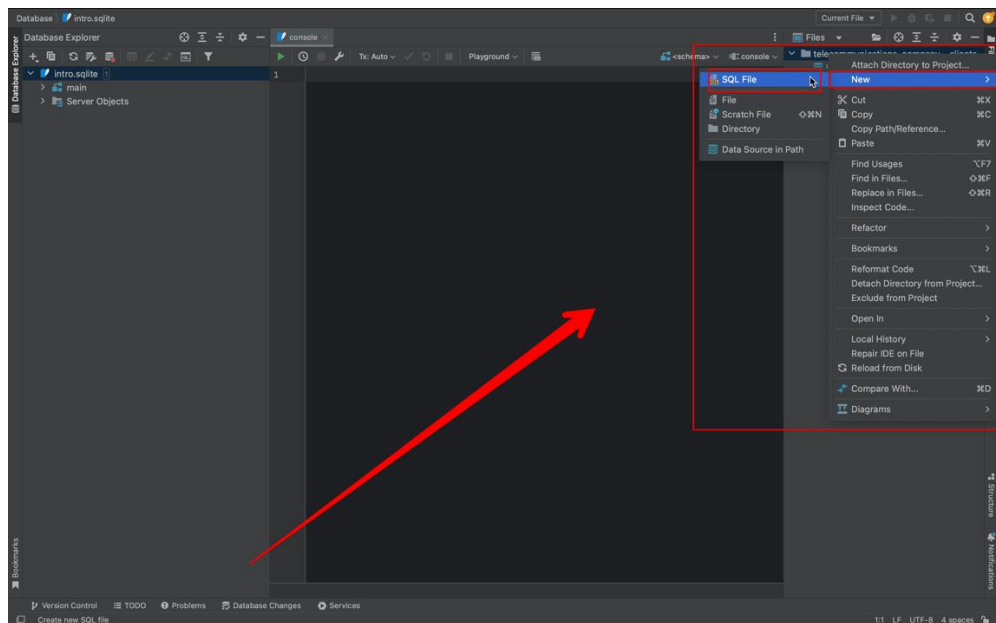


Далее даем название для базы данных, с расширением “.sqlite”

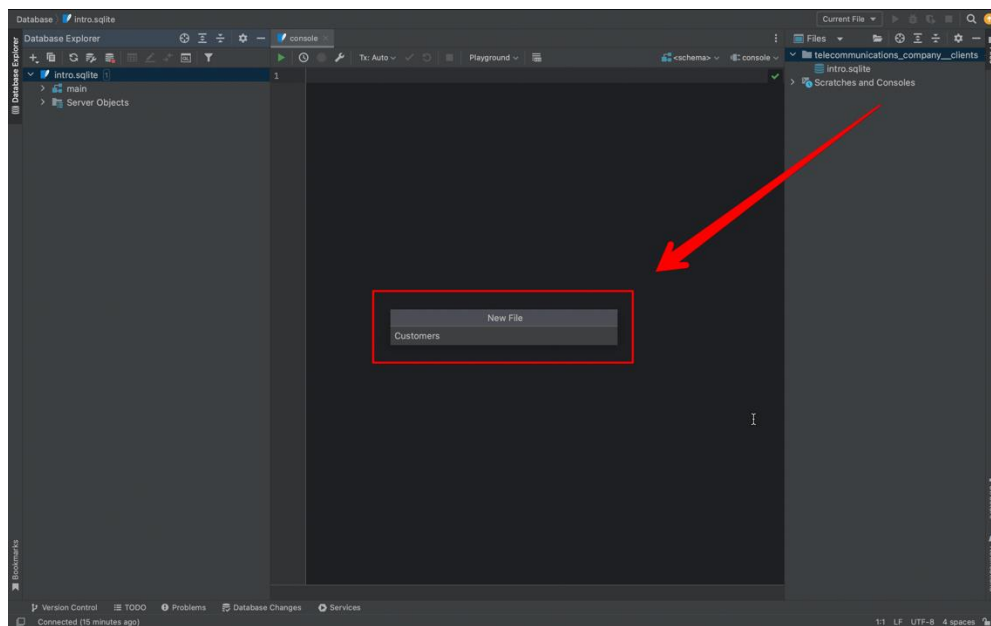


## 2.3 Создание таблиц

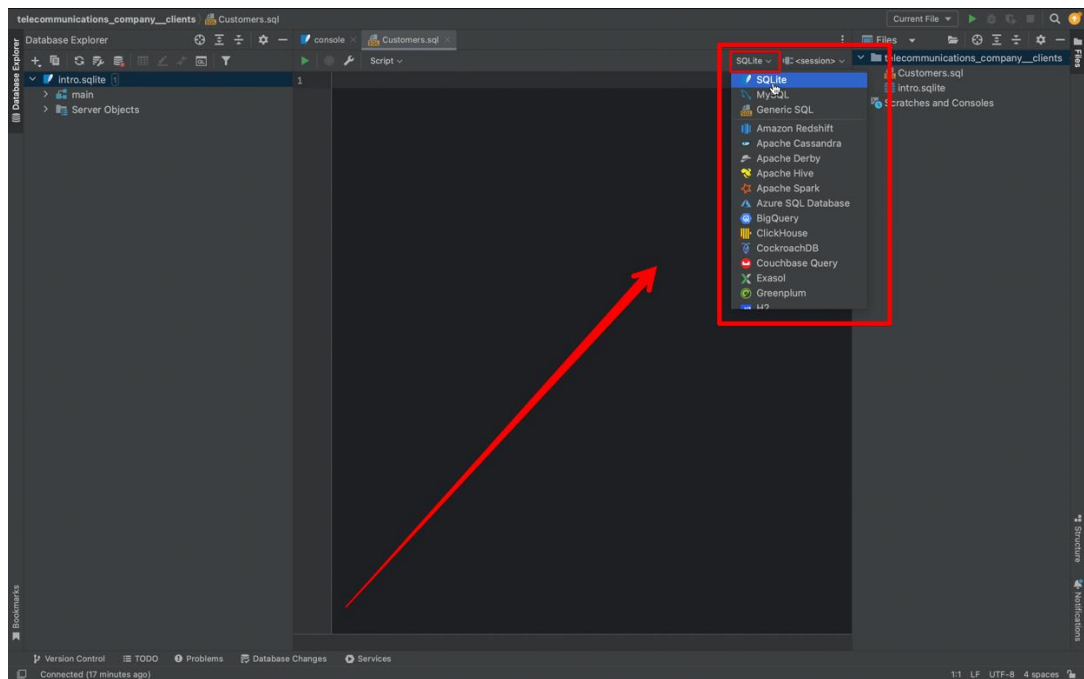
Справа в меню кликаем правой кнопкой мышки на папку проекта, и на появившемся контекстном меню наводим мышку на New, далее выбираем SQL File.



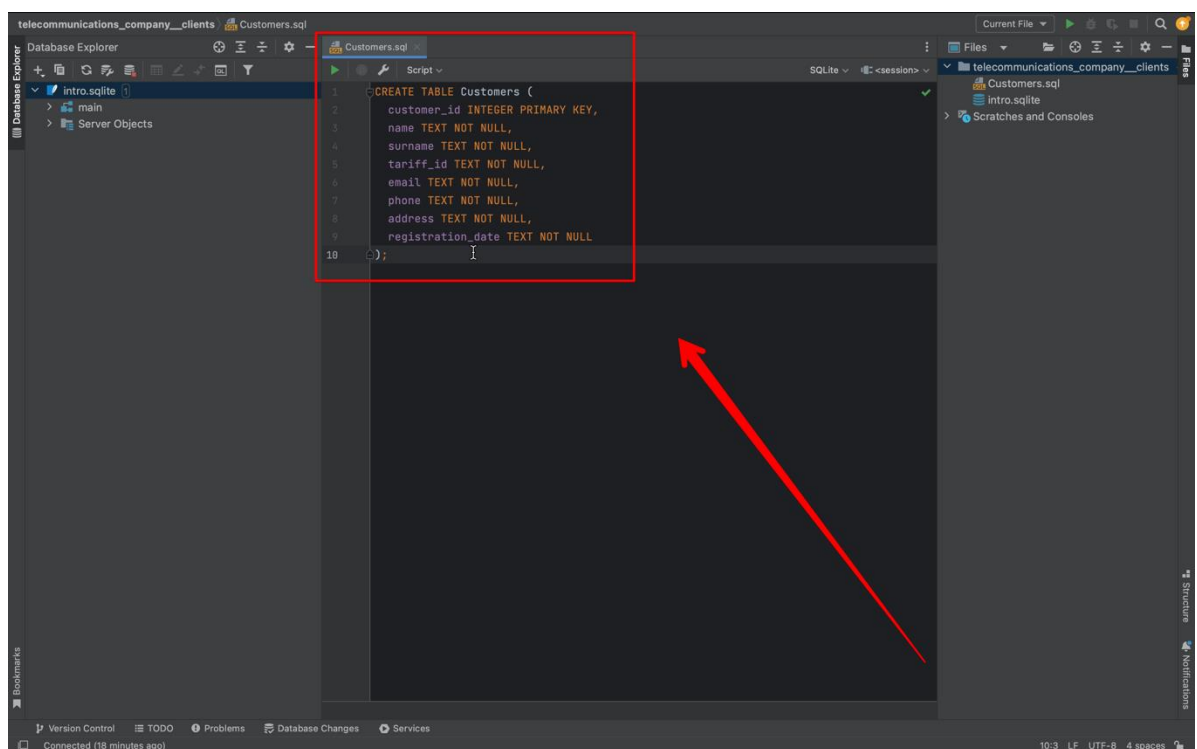
**Даем название для первой таблицы**



**Определяем наш синтаксис языка, выбираем SQLite**



В файле Customers создаем таблицу



Создание сеанса для таблицы

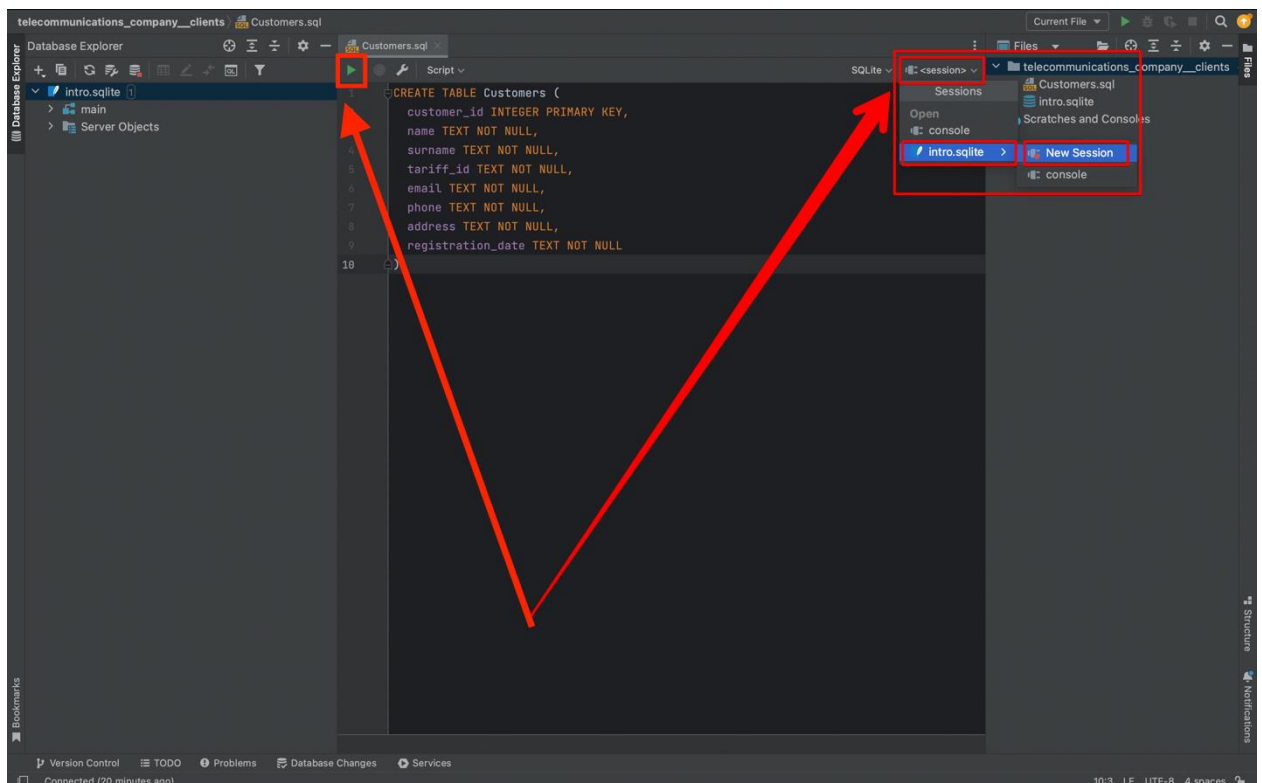
Когда вы подключаетесь к серверу базы данных из IDE, DataGrip и сервер устанавливают сеанс для обмена информацией. Сеанс представляет собой оболочку, в которой хранится информация о соединении (подключено

или отключено), управлении транзакциями (автоматическом или ручном), состоянии DBMS\_OUTPUT для Oracle (включено или отключено) и других настройках.

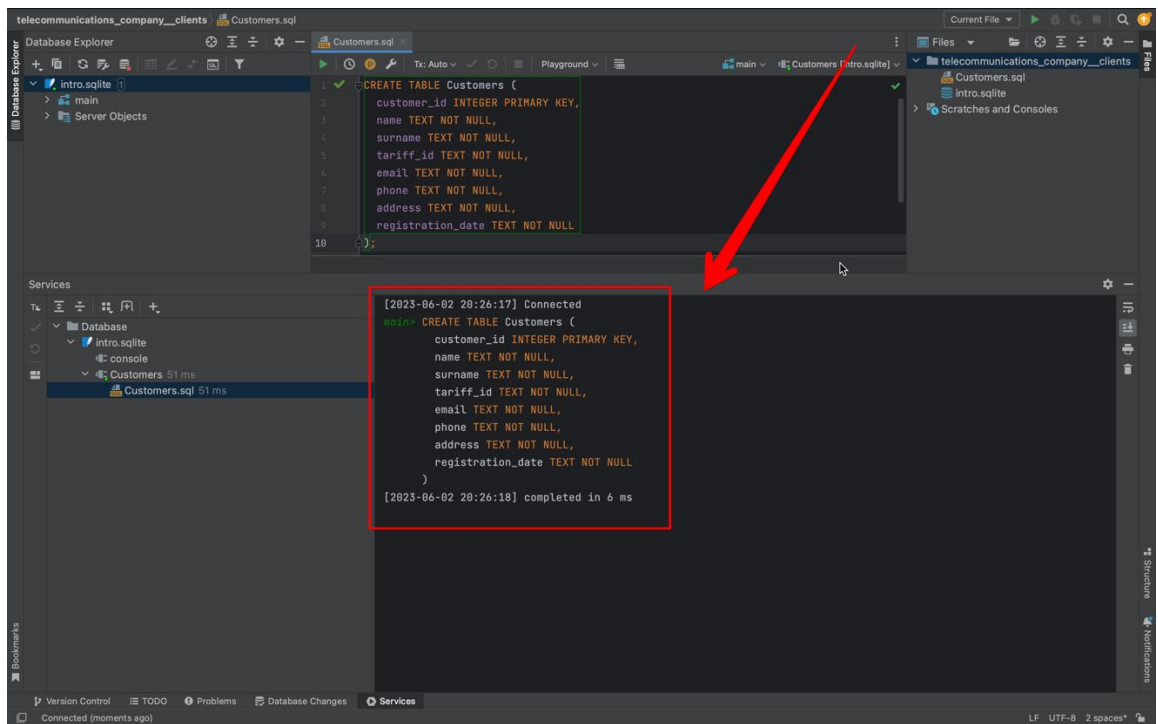
Сеанс имеет два состояния: подключен и отключен. Когда вы выполняете запрос к базе данных или обновляете состояние объекта, сеанс подключается. Маленький зеленый кружок указывает на то, что связь между IDE и источником данных активна.

В верхней панели нажимаем на <session> → intro.sqlite → New Session

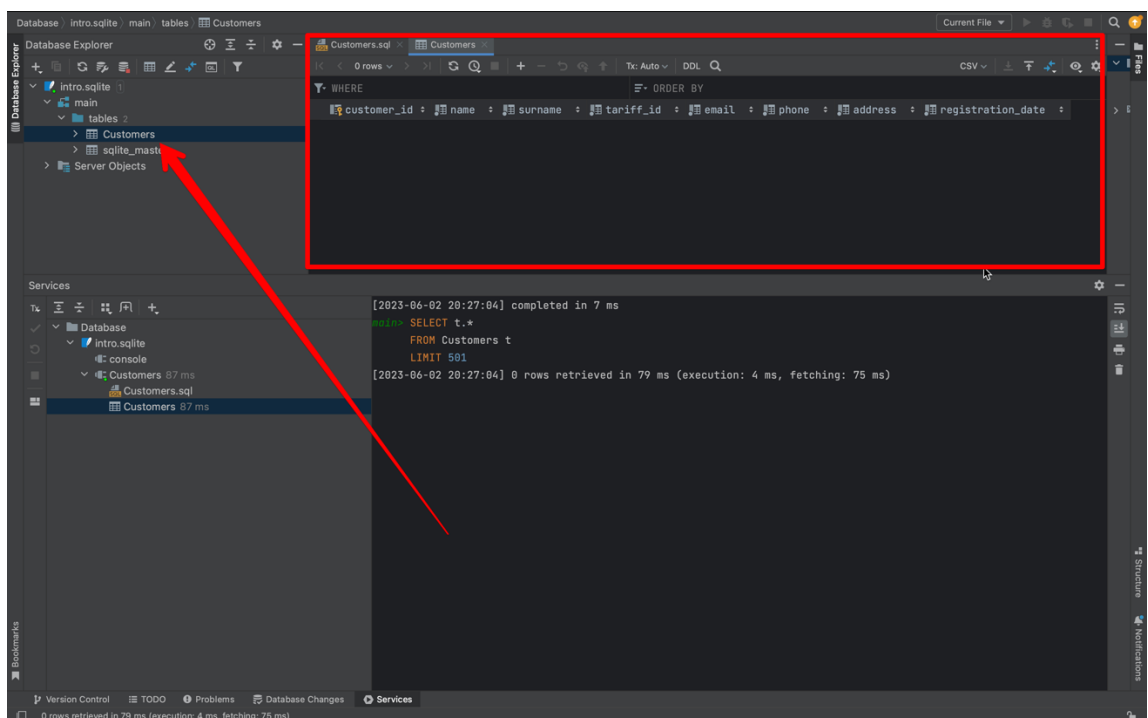
Далее запускаем наш код с помощью кнопки запуска которая находится на верхней панели.



После запуска у нас в проекте создастся папка “tables”, где будут храниться все таблицы, которые будут дальнейшем в проекте созданы.

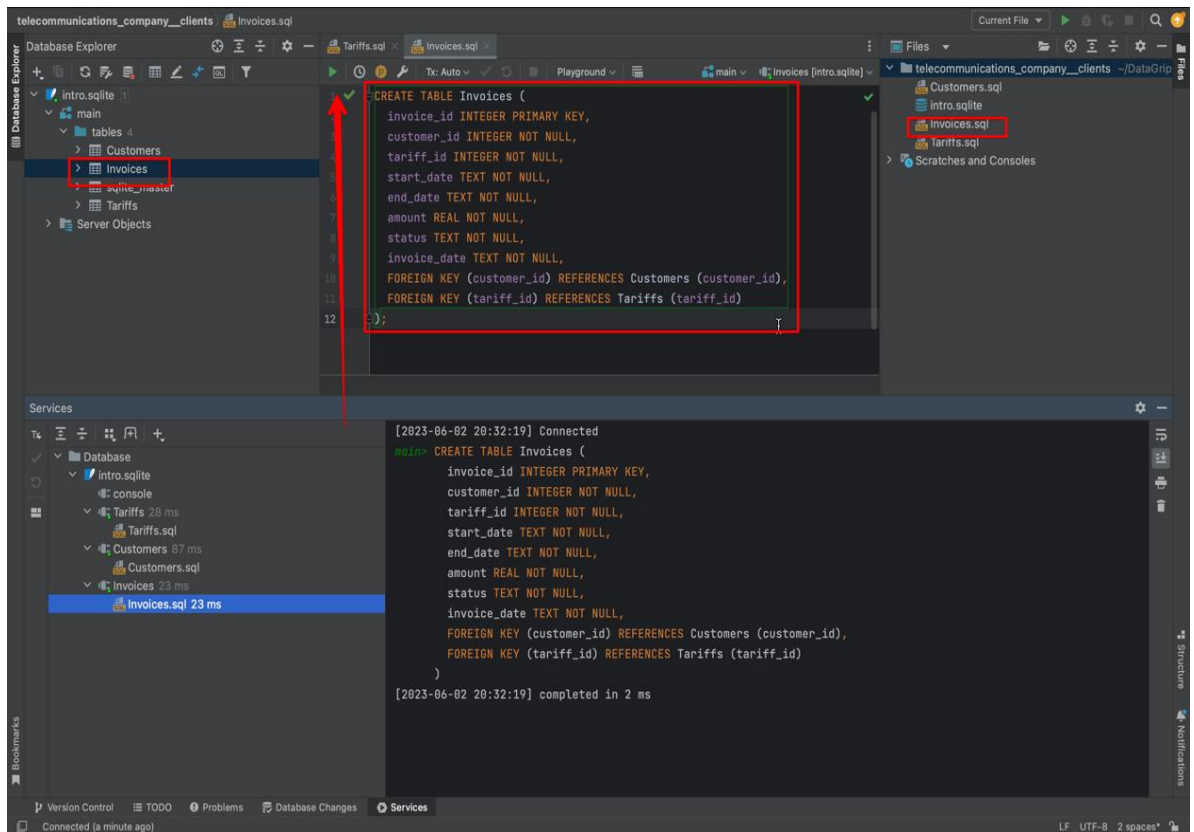
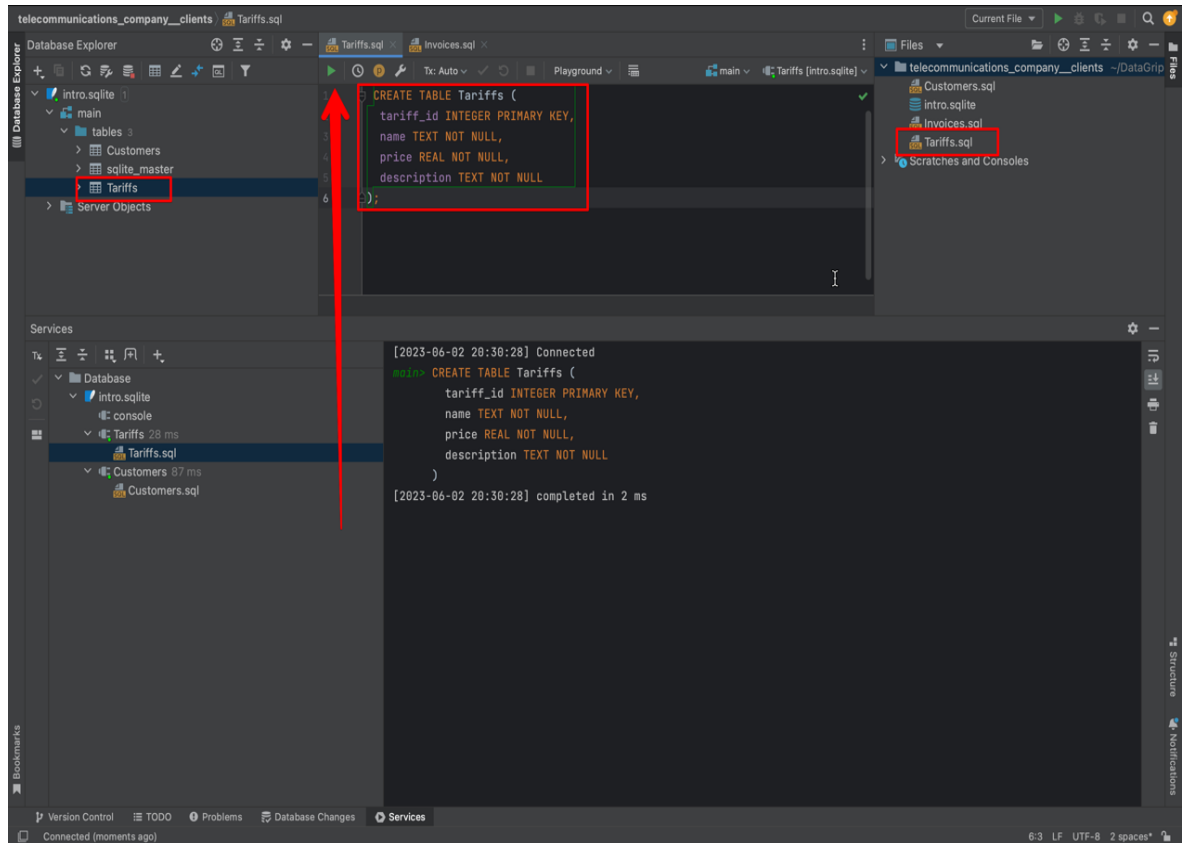


Мы можем открыть таблицу, и убедиться, что наша была успешно создана.



*Создание остальных таблиц*

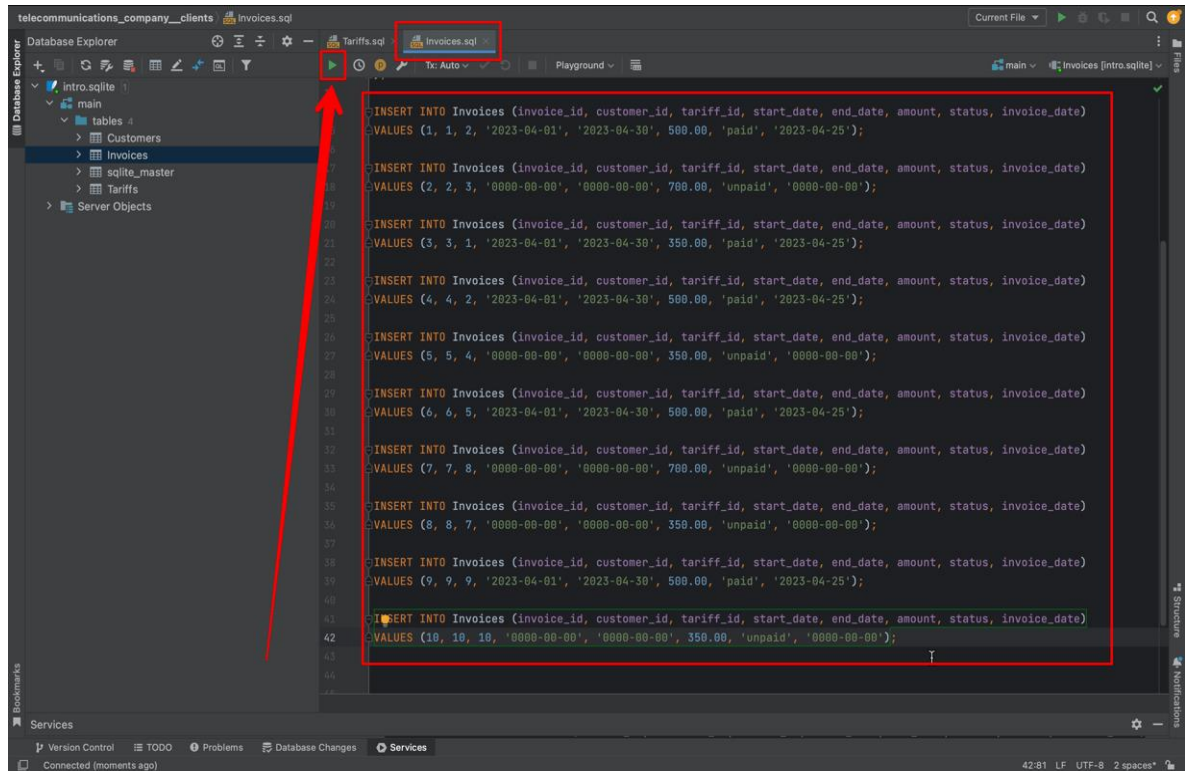
В таком же порядке создаем оставшиеся таблицы.



## 2.4 Заполнение таблиц

В этих же файлах (в таблицах) заполняем таблицу случайными данными

### Заполнение таблицы (Клиенты - Customers)





Database: intro.sqlite | main | tables | Invoices

Database Explorer: intro.sqlite | main | tables | Invoices

WHERE: invoice\_id : customer\_id : tariff\_id : start\_date : end\_date : amount : status : invoice\_date

| invoice_id | customer_id | tariff_id | start_date    | end_date   | amount | status | invoice_date |
|------------|-------------|-----------|---------------|------------|--------|--------|--------------|
| 1          | 1           | 1         | 2023-04-01    | 2023-04-30 | 500    | paid   | 2023-04-25   |
| 2          | 2           | 2         | 0000-00-00    | 0000-00-00 | 700    | unpaid | 0000-00-00   |
| 3          | 3           | 3         | 1 2023-04-01  | 2023-04-30 | 350    | paid   | 2023-04-25   |
| 4          | 4           | 4         | 2 2023-04-01  | 2023-04-30 | 500    | paid   | 2023-04-25   |
| 5          | 5           | 5         | 4 0000-00-00  | 0000-00-00 | 350    | unpaid | 0000-00-00   |
| 6          | 6           | 6         | 5 2023-04-01  | 2023-04-30 | 500    | paid   | 2023-04-25   |
| 7          | 7           | 7         | 8 0000-00-00  | 0000-00-00 | 700    | unpaid | 0000-00-00   |
| 8          | 8           | 8         | 7 0000-00-00  | 0000-00-00 | 350    | unpaid | 0000-00-00   |
| 9          | 9           | 9         | 9 2023-04-01  | 2023-04-30 | 500    | paid   | 2023-04-25   |
| 10         | 10          | 10        | 10 0000-00-00 | 0000-00-00 | 350    | unpaid | 0000-00-00   |

Services: [2023-06-02 20:36:13] completed in 6 ms

```
main> SELECT t.*
FROM Invoices t
LIMIT 501
```

[2023-06-02 20:36:13] 10 rows retrieved starting from 1 in 27 ms (execution: 3 ms, fetching: 24 ms)

## Заполнение таблицы (Тарифы - Tariffs)

telecommunications\_company\_clients | Tariffs.sql

Database Explorer: intro.sqlite | main | tables | Tariffs

WHERE: tariff\_id : name : price : description

| tariff_id | name           | price | description                                                                                |
|-----------|----------------|-------|--------------------------------------------------------------------------------------------|
| 1         | 'Стандарт'     | 250   | 'Базовый тариф с ограниченным количеством минут и трафика'                                 |
| 2         | 'Комфорт'      | 400   | 'Увеличенное количество минут и трафика'                                                   |
| 3         | 'Максимум'     | 600   | 'Неограниченное количество минут и трафика'                                                |
| 4         | 'Супер'        | 800   | 'Ультимативный тариф с неограниченным количеством минут, трафика и подарками каждый месяц' |
| 5         | 'Свобода'      | 350   | 'Тариф с возможностью изменения состава услуг в любое время'                               |
| 6         | 'Безлимит'     | 500   | 'Тариф с неограниченным интернетом и минутами'                                             |
| 7         | 'Интернет-M'   | 200   | 'Тариф с ограниченным количеством минут, но большим трафиком'                              |
| 8         | 'Интернет-L'   | 350   | 'Тариф с ограниченным количеством минут и большим трафиком'                                |
| 9         | 'Домашний'     | 300   | 'Тариф для домашнего использования с большим трафиком'                                     |
| 10        | 'Путешествие'  | 100   | 'Тариф для использования за границей с минимальными расходами'                             |
| 11        | 'Аренда'       | 500   | 'Тариф для аренды телефона в других странах'                                               |
| 12        | 'Детский'      | 200   | 'Тариф для детей до 12 лет с ограниченным контентом'                                       |
| 13        | 'VIP'          | 1500  | 'Эксклюзивный тариф для VIP-клиентов'                                                      |
| 14        | 'Друзья'       | 200   | 'Тариф для общения с друзьями и близкими'                                                  |
| 15        | 'Студенческий' | 300   | 'Тариф для студентов с особыми условиями'                                                  |

Services: main> INSERT INTO Tariffs (tariff\_id, name, price, description)

```
VALUES
(1, 'Стандарт', 250, 'Базовый тариф с ограниченным количеством минут и трафика'),
(2, 'Комфорт', 400, 'Увеличенное количество минут и трафика'),
(3, 'Максимум', 600, 'Неограниченное количество минут и трафика'),
(4, 'Супер', 800, 'Ультимативный тариф с неограниченным количеством минут, трафика и подарками каждый месяц'),
(5, 'Свобода', 350, 'Тариф с возможностью изменения состава услуг в любое время'),
(6, 'Безлимит', 500, 'Тариф с неограниченным интернетом и минутами'),
(7, 'Интернет-M', 200, 'Тариф с ограниченным количеством минут, но большим трафиком'),
(8, 'Интернет-L', 350, 'Тариф с ограниченным количеством минут и большим трафиком'),
(9, 'Домашний', 300, 'Тариф для домашнего использования с большим трафиком'),
(10, 'Путешествие', 100, 'Тариф для использования за границей с минимальными расходами'),
(11, 'Аренда', 500, 'Тариф для аренды телефона в других странах'),
(12, 'Детский', 200, 'Тариф для детей до 12 лет с ограниченным контентом'),
(13, 'VIP', 1500, 'Эксклюзивный тариф для VIP-клиентов'),
(14, 'Друзья', 200, 'Тариф для общения с друзьями и близкими'),
(15, 'Студенческий', 300, 'Тариф для студентов с особыми условиями');
```

[2023-06-02 20:36:13] 15 rows retrieved starting from 1 in 18 ms (execution: 2 ms, fetching: 16 ms)



Database: intro.sqlite | main | tables | Tariffs

Database Explorer: intro.sqlite | main | tables | Tariffs

Table Data:

| tariff_id | name         | price | description                                      |
|-----------|--------------|-------|--------------------------------------------------|
| 1         | Стандарт     | 250   | Базовый тариф с ограниченным количеством мину... |
| 2         | Комфорт      | 400   | Увеличенное количество минут и трафика           |
| 3         | Максимум     | 600   | Неограниченное количество минут и трафика        |
| 4         | Супер        | 800   | Ультимативный тариф с неограниченным количест... |
| 5         | Свобода      | 350   | Тариф с возможностью изменения состава услуг -   |
| 6         | Безлимит     | 500   | Тариф с неограниченным интернетом и минутами     |
| 7         | Интернет-М   | 200   | Тариф с ограниченным количеством минут, но бо... |
| 8         | Интернет-L   | 350   | Тариф с ограниченным количеством минут и боль... |
| 9         | Домашний     | 300   | Тариф для домашнего использования с большим т... |
| 10        | Путешествие  | 100   | Тариф для использования за границей с минимал... |
| 11        | Аренда       | 500   | Тариф для аренды телефона в других странах       |
| 12        | Детский      | 200   | Тариф для детей до 12 лет с ограниченным конт... |
| 13        | VIP          | 1500  | Эксклюзивный тариф для VIP-клиентов              |
| 14        | Друзья       | 200   | Тариф для общения с друзьями и близкими          |
| 15        | Студенческий | 300   | Тариф для студентов с особыми условиями          |

Console: [2023-06-02 20:40:45] completed in 8 ms  
 SELECT t.\*  
 FROM Tariffs t  
 LIMIT 501  
 [2023-06-02 20:40:45] 15 rows retrieved starting from 1 in 18 ms (execution: 2 ms, fetching: 16 ms)

## Заполнение таблицы (Счета - Invoices).

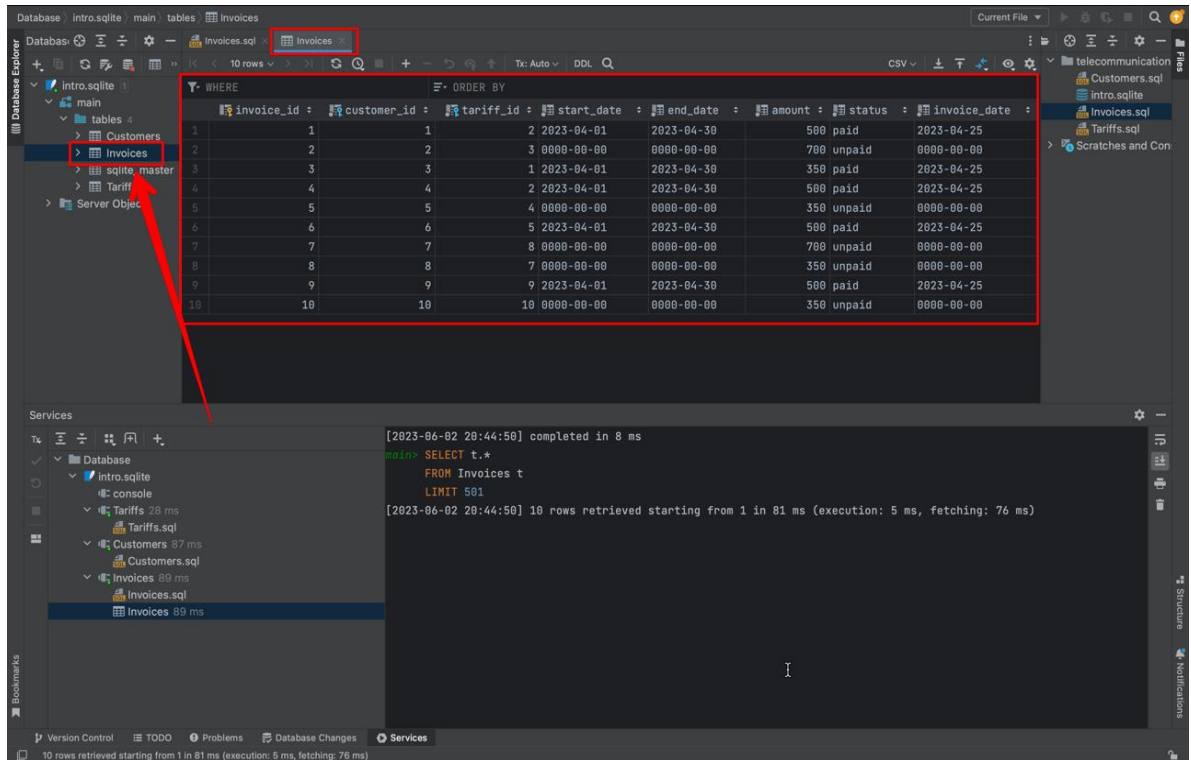
Database: intro.sqlite | main | tables | Invoices

Database Explorer: intro.sqlite | main | tables | Invoices

Table Data:

| invoice_id | customer_id | tariff_id | start_date   | end_date     | amount | status   | invoice_date |
|------------|-------------|-----------|--------------|--------------|--------|----------|--------------|
| 1          | 1           | 2         | '2023-04-01' | '2023-04-30' | 500.00 | 'paid'   | '2023-04-25' |
| 2          | 2           | 3         | '0000-00-00' | '0000-00-00' | 700.00 | 'unpaid' | '0000-00-00' |
| 3          | 3           | 1         | '2023-04-01' | '2023-04-30' | 350.00 | 'paid'   | '2023-04-25' |
| 4          | 4           | 2         | '2023-04-01' | '2023-04-30' | 500.00 | 'paid'   | '2023-04-25' |
| 5          | 5           | 4         | '0000-00-00' | '0000-00-00' | 350.00 | 'unpaid' | '0000-00-00' |
| 6          | 6           | 5         | '2023-04-01' | '2023-04-30' | 500.00 | 'paid'   | '2023-04-25' |
| 7          | 7           | 8         | '0000-00-00' | '0000-00-00' | 700.00 | 'unpaid' | '0000-00-00' |
| 8          | 8           | 7         | '0000-00-00' | '0000-00-00' | 350.00 | 'unpaid' | '0000-00-00' |
| 9          | 9           | 9         | '2023-04-01' | '2023-04-30' | 500.00 | 'paid'   | '2023-04-25' |
| 10         | 10          | 10        | '0000-00-00' | '0000-00-00' | 350.00 | 'unpaid' | '0000-00-00' |

Console: INSERT INTO Invoices (invoice\_id, customer\_id, tariff\_id, start\_date, end\_date, amount, status, invoice\_date)  
 VALUES (1, 1, 2, '2023-04-01', '2023-04-30', 500.00, 'paid', '2023-04-25');  
 [2023-06-02 20:34:42] 1 row affected in 1 ms

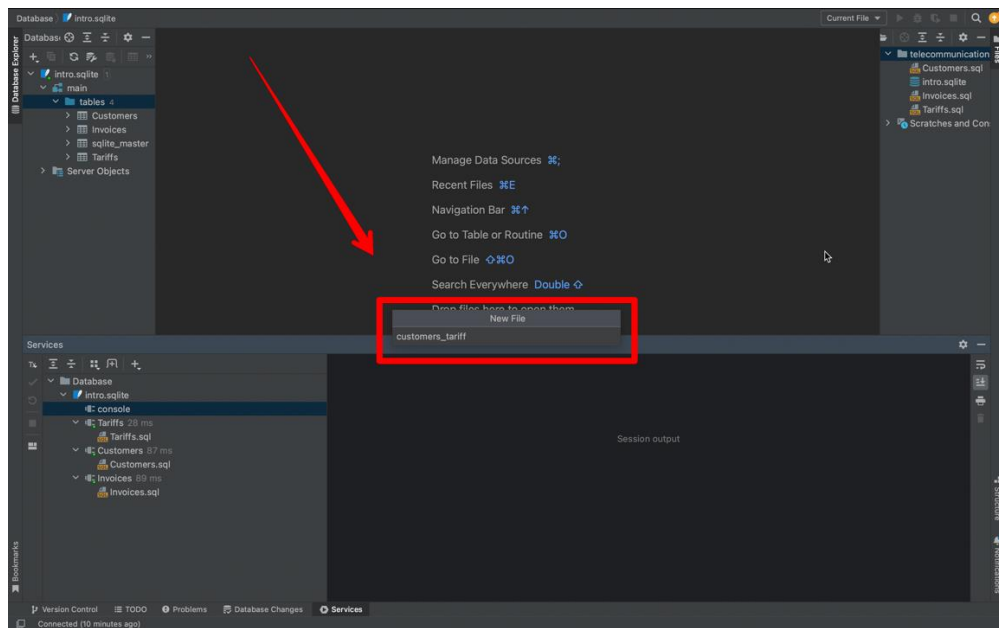


## 2.5 Создание запросов

Как создавать запросы?

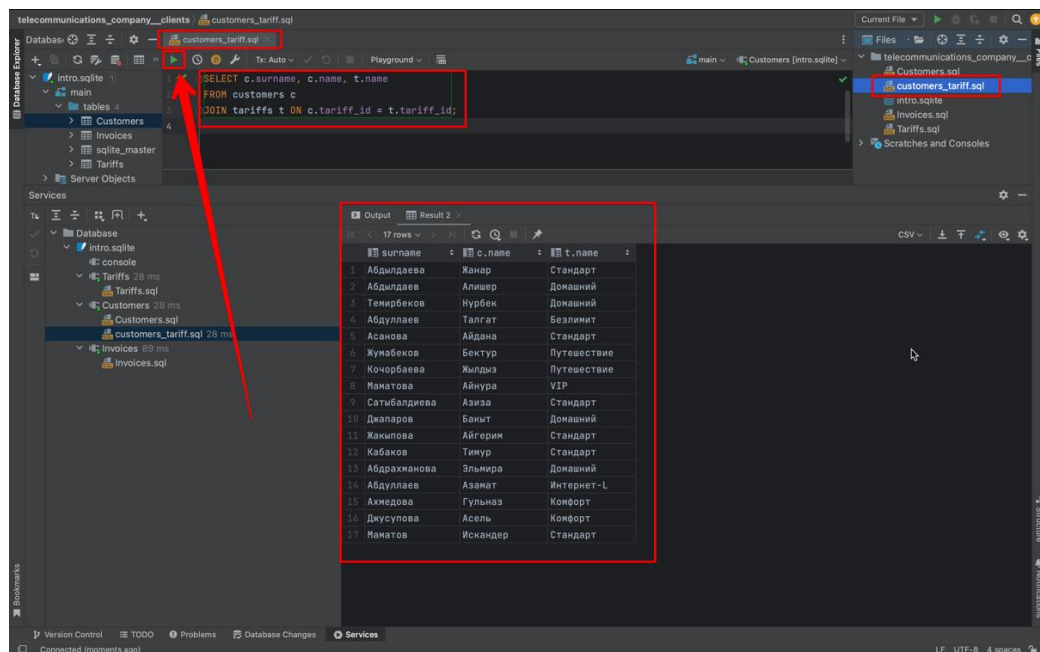
Для начала создаем новый SQL файл,

Даем название запросу (файлу)



В файле пишем сам запрос (любой).

Затем нажимаем на треугольную, зеленую кнопку слева сверху

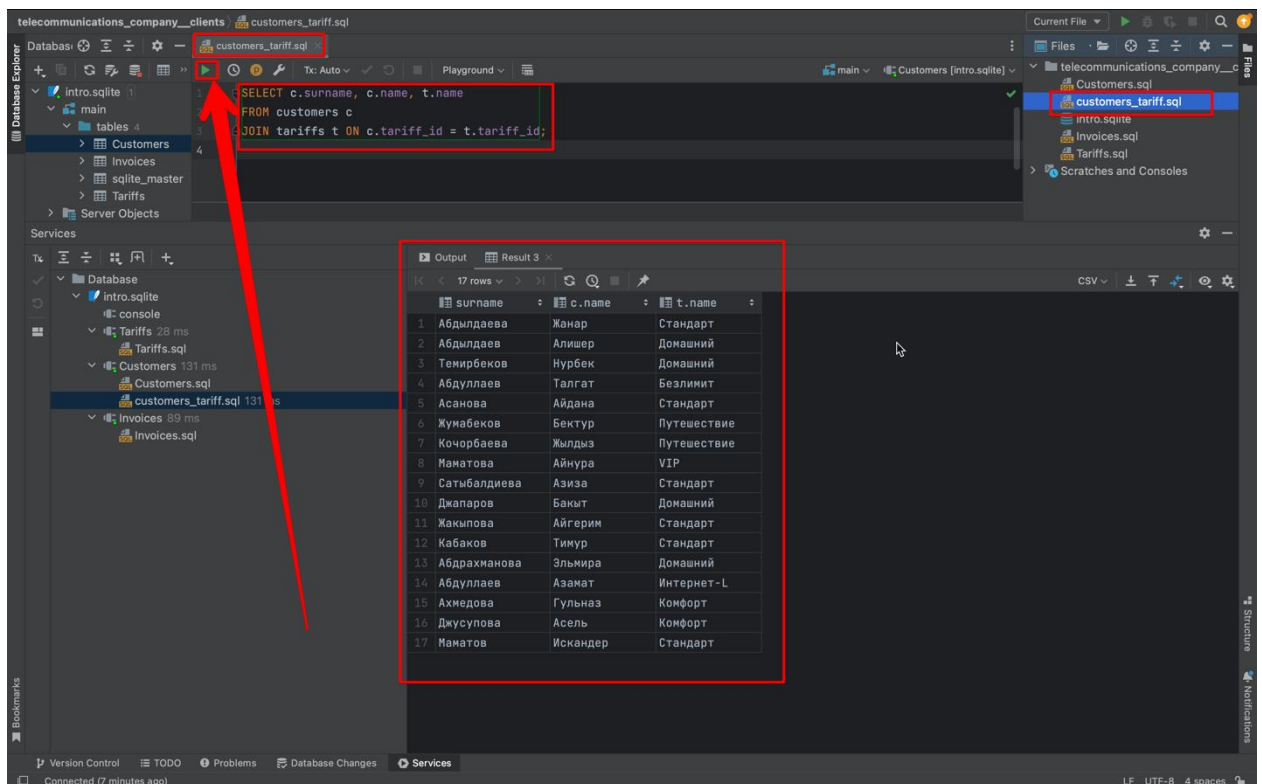


## Создание запроса “customers\_tariff”.

Следующий запрос предназначен для получения списка клиентов, и списка тарифов которые подключены к отдельно взятому клиенту.

В этом запросе мы выбираем столбцы **surname** и **name** из таблицы **Customers**, и столбец **name** из таблицы **Tariffs**. Мы используем оператор

**JOIN** для объединения таблиц на основе ключей **tariff\_id** из таблицы **Tariffs** и **tariff\_id** из таблицы **Customers**.



### *Создание запроса "invoices\_status".*

Следующий запрос предназначен для получения списка клиентов, которые не внесли платеж на интернет.

В этом запросе мы выбираем столбцы **surname**, **name** и **registration\_date** из таблицы **Customers**. И столбцы **start\_date**, **end\_date**, **amount** и **status** из таблицы **Invoices**. Затем мы столбец **status** приравниваем к значению **unpaid**, чтобы вывести список клиентов, которые не оплатили за интернет. Здесь тоже мы используем оператор **JOIN** для объединения таблиц на основе ключей **customer\_id** из таблицы **Customers** и **customer\_id** из таблицы **Invoices**.

The screenshot shows a database IDE with the following components:

- Database Explorer:** Shows a project structure with folders like 'intro.sqlite', 'main', 'tables', 'Customers', 'Invoices', 'sqlite\_master', 'sqlite\_sequence', and 'Tariffs'.
- SQL Editor:** Contains the following query:
 

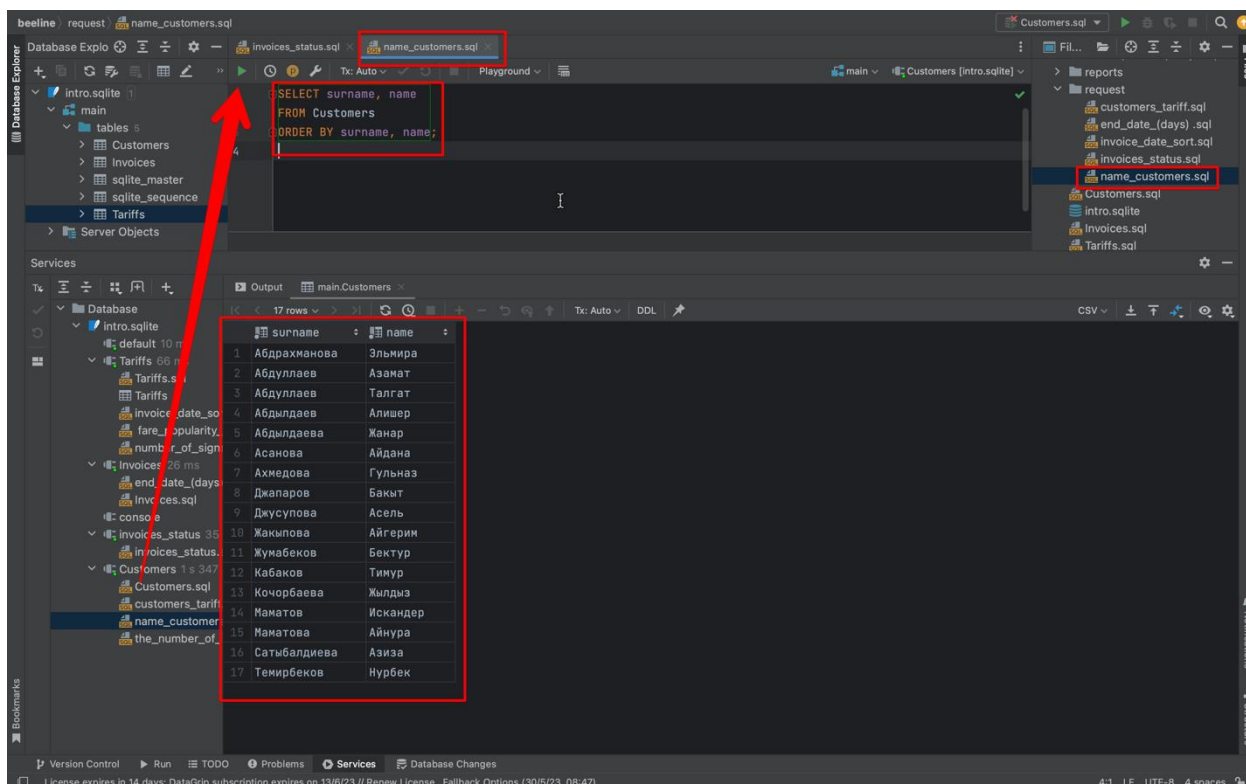
```
SELECT I.status == 'unpaid', C.surname, C.name, C.registration_date, I.start_date, I.end_date, I.amount
FROM Customers C
JOIN Invoices I on C.customer_id = I.customer_id
```
- Output Panel:** Displays the results of the query in a table format. The first column is a boolean expression, and the others are customer and invoice details.
 

| "I.status == 'unpaid'" | surname      | name   | registration_date | start_date | end_date   | amount |
|------------------------|--------------|--------|-------------------|------------|------------|--------|
| 0                      | Абдылдаева   | Жанар  | 2022-01-15        | 2023-04-01 | 2023-04-30 | 500    |
| 1                      | Абдылдаев    | Алишер | 2021-12-10        | 2023-04-01 | 2023-04-30 | 700    |
| 0                      | Темирбеков   | Нурбек | 2023-03-01        | 2023-04-01 | 2023-04-30 | 350    |
| 0                      | Абдуллаев    | Талгат | 2021-02-28        | 2023-04-01 | 2023-04-30 | 500    |
| 1                      | Асанова      | Айдана | 2023-05-03        | 2023-04-01 | 2023-04-30 | 350    |
| 0                      | Жунабеков    | Бектур | 2020-10-25        | 2023-04-01 | 2023-04-30 | 500    |
| 1                      | Кочорбаева   | Жылдыз | 2022-09-17        | 2023-04-01 | 2023-04-30 | 700    |
| 1                      | Маматова     | Айнура | 2021-11-05        | 2023-04-01 | 2023-04-30 | 350    |
| 0                      | Сатыбалдиева | Азиза  | 2022-07-11        | 2023-04-01 | 2023-04-30 | 500    |
| 1                      | Джапаров     | Бакыт  | 2020-12-28        | 2023-04-01 | 2023-04-30 | 350    |

## Создание запроса “name\_customers”.

Следующий запрос предназначен для получения списка фамилии и имен клиентов по алфавитному порядку.

В этом запросе мы выбираем столбцы **surname** и **name** из таблицы **Customers**. Здесь мы используем оператор **ORDER BY** для вывода данных по алфавитному порядку.

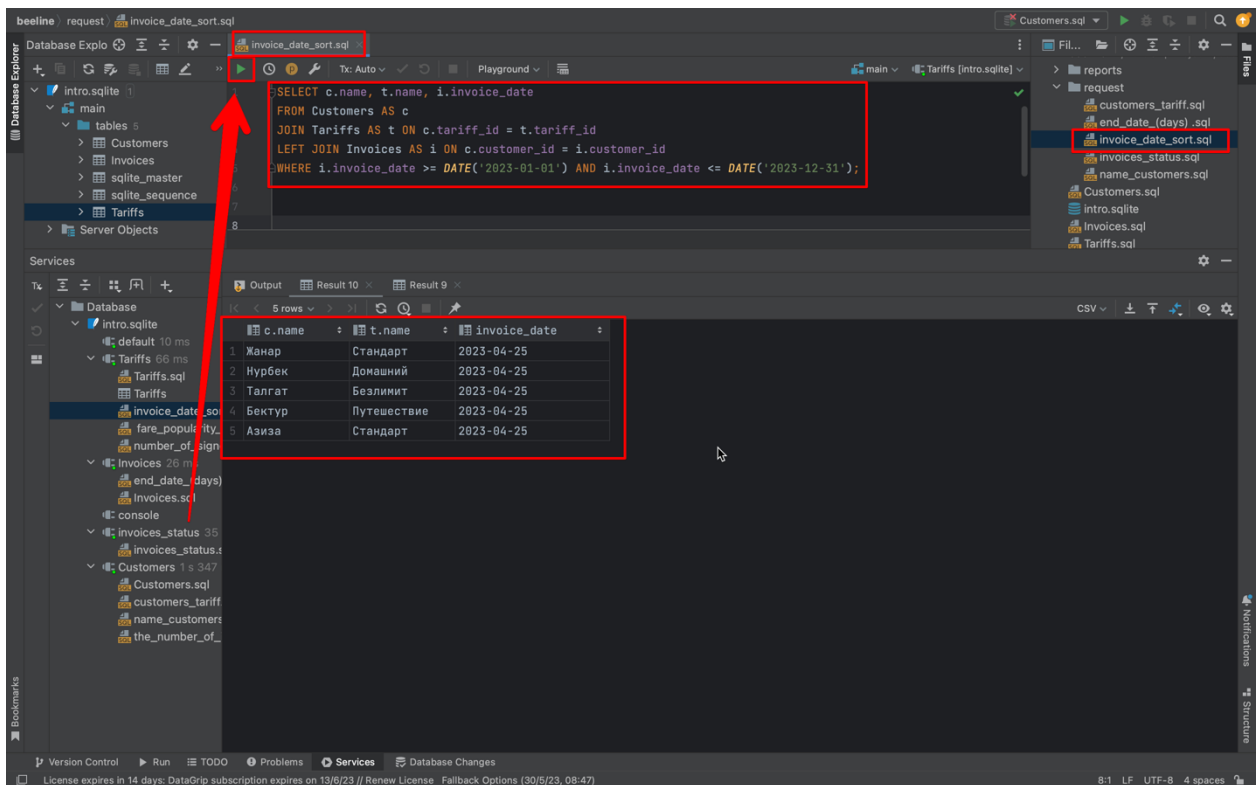


## Создание запроса “invoices\_date\_sort”.

Следующий запрос предназначен для получения списка фамилии и имен клиентов у которых даты оплаты находятся в диапазоне от 1 января 2023 года до 31 декабря 2023 года.

В этом запросе мы выбираем столбцы **customer\_name** из таблицы **Customers**, **name** из таблицы **Tariffs** и **invoice\_date** из таблицы **Invoices**. Мы используем оператор **JOIN** для объединения таблиц на основе ключей **tariff\_id** и **customer\_id**, а также оператор **LEFT JOIN** для включения всех записей из таблицы **Customers**, даже если нет соответствующих записей в таблице **Invoices**. Затем мы добавляем условие **WHERE**, чтобы выбрать только счета, даты которых находятся в диапазоне от 1 января 2023 года до 31 декабря 2023 года.

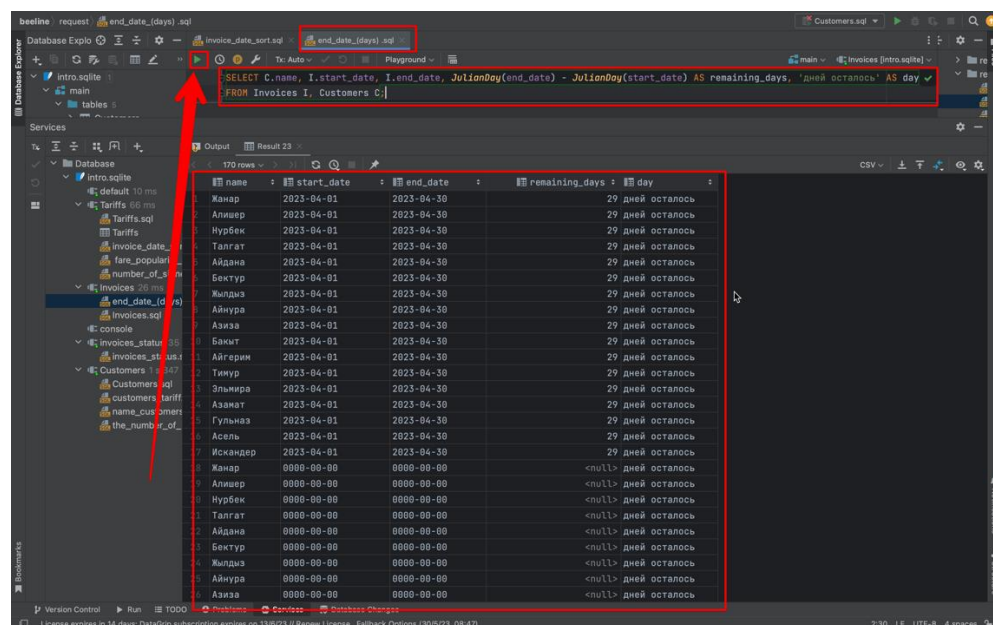




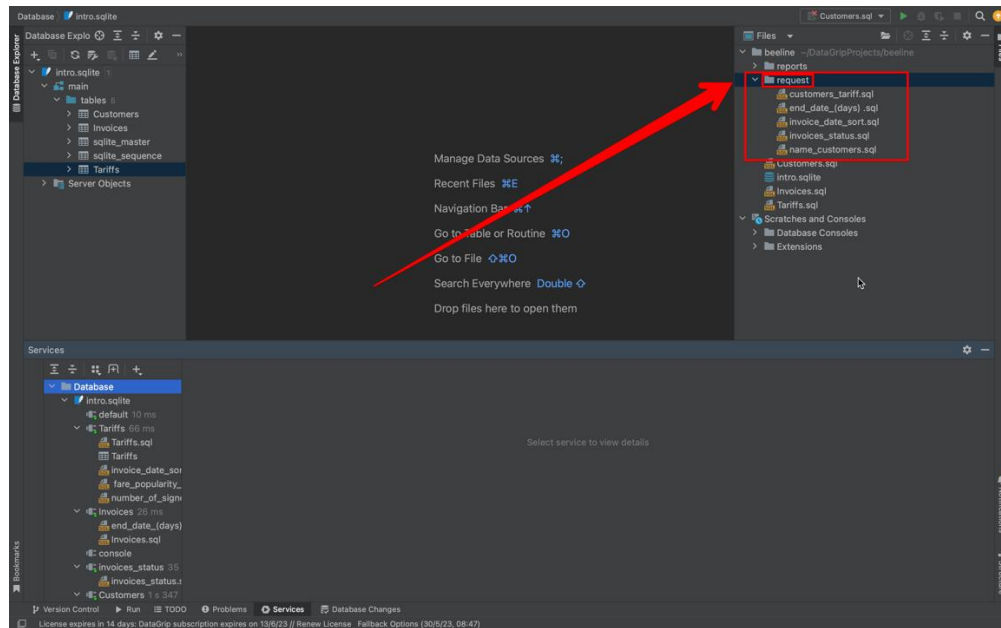
## Создание запроса “end\_date\_(days)”

Следующий запрос предназначен для вывода дней до окончания срока.

В этом запросе мы используем функцию **JulianDay()** для вычисления числа дней между датой окончания **end\_date** и текущей датой **'now'**. Разница в днях будет представлена в столбце **remaining\_days**.



В конце создания всех запросов, я решил создать отдельную папку для запросов, назвал её “request”.



## 2.6 Создание отчетов

Создание отчетов на основе запросов, позволяющих удобно просматривать и анализировать данные.

Этот запрос вычисляет количество подписанных тарифов для каждого клиента на основе таблиц "Tariffs", "Customers" и "Invoices". Результаты отсортированы по убыванию количества подписанных тарифов.



beeline reports number\_of\_signed\_tariffs\_for\_each\_client.sql

```
SELECT c.name, COUNT(DISTINCT t.tariff_id) AS subscribed_tariffs
FROM Customers c
LEFT JOIN Invoices i ON c.customer_id = i.customer_id
LEFT JOIN Tariffs t ON i.tariff_id = t.tariff_id
GROUP BY c.customer_id, c.name
ORDER BY subscribed_tariffs DESC;
```

| name        | subscribed_tariffs |
|-------------|--------------------|
| 1 Жанар     | 1                  |
| 2 Алишер    | 1                  |
| 3 Нурбек    | 1                  |
| 4 Талгат    | 1                  |
| 5 Айдана    | 1                  |
| 6 Бектур    | 1                  |
| 7 Жылдыз    | 1                  |
| 8 Айнура    | 1                  |
| 9 Азиза     | 1                  |
| 10 Бакыт    | 1                  |
| 11 Айгерим  | 0                  |
| 12 Тимур    | 0                  |
| 13 Эльмира  | 0                  |
| 14 Азамат   | 0                  |
| 15 Гульназ  | 0                  |
| 16 Асель    | 0                  |
| 17 Искандер | 0                  |

Этот запрос вычисляет количество выставленных счетов и общую сумму счетов для каждого клиента из таблиц "Customers" и "Invoices". Результаты отсортированы по убыванию общей суммы счетов.

beeline reports the\_number\_of\_invoices\_issued\_and\_the\_total\_amount.sql

```
SELECT c.customer_id, c.name, COUNT(i.invoice_id) AS invoice_count, SUM(i.amount) AS total_amount
FROM Customers c
LEFT JOIN Invoices i ON c.customer_id = i.customer_id
GROUP BY c.customer_id, c.name
ORDER BY total_amount DESC;
```

| customer_id | name | invoice_count | total_amount |
|-------------|------|---------------|--------------|
| 2 Алишер    |      | 1             | 700          |
| 7 Жылдыз    |      | 1             | 700          |
| 1 Жанар     |      | 1             | 500          |
| 4 Талгат    |      | 1             | 500          |
| 6 Бектур    |      | 1             | 500          |
| 9 Азиза     |      | 1             | 500          |
| 3 Нурбек    |      | 1             | 350          |
| 5 Айдана    |      | 1             | 350          |
| 8 Айнура    |      | 1             | 350          |
| 10 Бакыт    |      | 1             | 350          |
| 11 Айгерим  |      | 0             | <null>       |
| 12 Тимур    |      | 0             | <null>       |
| 13 Эльмира  |      | 0             | <null>       |
| 14 Азамат   |      | 0             | <null>       |
| 15 Гульназ  |      | 0             | <null>       |
| 16 Асель    |      | 0             | <null>       |
| 17 Искандер |      | 0             | <null>       |

Этот запрос вычисляет популярность тарифов на основе количества уникальных клиентов, использующих каждый тариф, и общей выручки,

полученной от каждого тарифа.

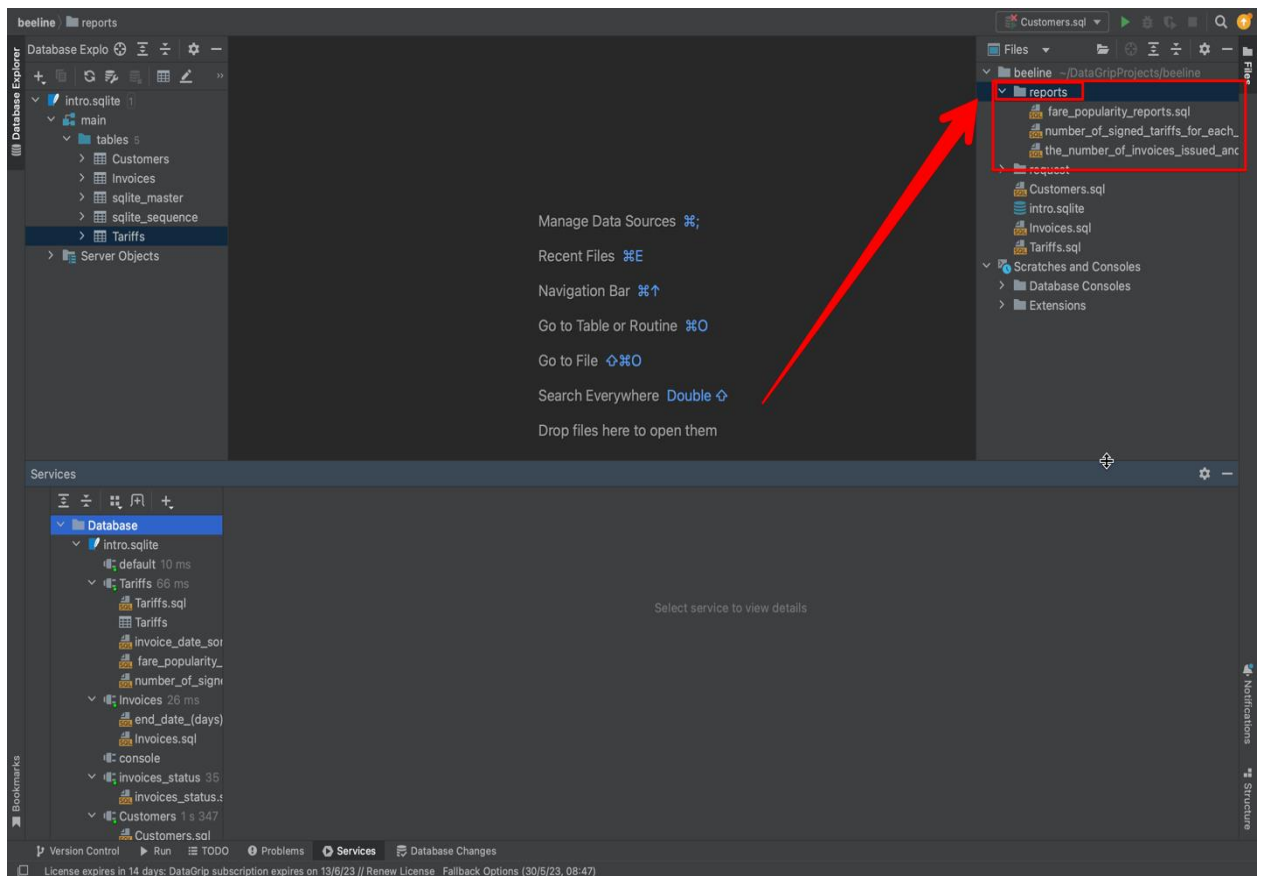
The screenshot shows the DataGrip IDE interface. The top pane displays the SQL query 'number\_of\_signed\_tariffs\_for\_each\_client.sql'. The query is as follows:

```
SELECT c.name, COUNT(DISTINCT t.tariff_id) AS subscribed_tariffs
FROM Customers c
LEFT JOIN Invoices i ON c.customer_id = i.customer_id
LEFT JOIN Tariffs t ON i.tariff_id = t.tariff_id
GROUP BY c.customer_id, c.name
ORDER BY subscribed_tariffs DESC;
```

The bottom pane shows the results of the query, which are 17 rows. The first two columns are 'name' and 'subscribed\_tariffs'. The results are as follows:

| name        | subscribed_tariffs |
|-------------|--------------------|
| 1 Жанар     | 1                  |
| 2 Алишер    | 1                  |
| 3 Нурбек    | 1                  |
| 4 Талгат    | 1                  |
| 5 Айдана    | 1                  |
| 6 Бектур    | 1                  |
| 7 Жылдыз    | 1                  |
| 8 Айнура    | 1                  |
| 9 Азиза     | 1                  |
| 10 Бакыт    | 1                  |
| 11 Айгерим  | 0                  |
| 12 Тимур    | 0                  |
| 13 Эльмира  | 0                  |
| 14 Азамат   | 0                  |
| 15 Гульназ  | 0                  |
| 16 Асель    | 0                  |
| 17 Искандер | 0                  |

A red arrow points from the file explorer on the left to the file 'number\_of\_signed\_tariffs\_for\_each\_client.sql' in the 'reports' folder.



Так же и тут я собрал все отчёты в одну папку

## ЗАКЛЮЧЕНИЕ

В рамках данной выпускной квалификационной работе была разработана и реализована база данных для подсистемы расчетов с клиентами в телекоммуникационной компании. База данных была построена с использованием SQLite и разработана с помощью интегрированной среды разработки DataGrip.

В базе данных было создано три таблицы: Customers, Tariffs и Invoices. Таблица Customers содержит информацию о клиентах, включая их идентификатор, имя, фамилию, электронную почту, телефон, адрес и дату регистрации. Таблица Tariffs содержит информацию о тарифных планах, включая их идентификатор, название, цену и описание. Таблица Invoices содержит информацию о счетах клиентов, включая их идентификатор, идентификатор клиента, идентификатор тарифного плана, сумму счета, дату выставления, дату оплаты и статус оплаты.

Кроме того, были разработаны пять отчетов, предоставляющих информацию о клиентах, тарифных планах и счетах. Отчеты позволяют получать сводные данные, статистику и аналитическую информацию, необходимую для управления расчетами с клиентами. Три отчета предоставляют различные фильтры и сортировки для более гибкого анализа данных.

В результате выполнения данной работы была успешно создана база данных и реализованы необходимые таблицы и отчеты. База данных позволяет эффективно хранить и управлять информацией о клиентах, тарифных планах и счетах, а отчеты обеспечивают ценную аналитическую информацию для принятия управленческих решений.

Работа с базой данных была выполнена с использованием SQLite и интегрированной среды разработки DataGrip, что позволило удобно проектировать, создавать и управлять базой данных. DataGrip предоставляет

широкий набор функций и инструментов для разработки баз данных, обеспечивая удобство работы и повышая производительность.

Таким образом, разработка базы данных и отчетов для подсистемы расчетов с клиентами в телекоммуникационной компании позволяет улучшить эффективность работы с данными, повысить уровень сервиса и принимать обоснованные решения на основании надежной информации. База данных на SQLite обладает высокой производительностью и надежностью, что делает ее идеальным выбором для подобных приложений.

В ходе выполнения данной работы было проведено тестирование базы данных и отчетов для подсистемы расчетов с клиентами. Были проверены функции добавления, обновления и удаления данных, а также выполнение сложных запросов для получения требуемой информации. Результаты тестирования показали правильную работу базы данных и отчетов, что подтверждает их функциональность и соответствие поставленным требованиям.

Однако, как любой другой проект, данная система имеет потенциал для дальнейшего улучшения. Возможные направления развития включают расширение функционала отчетов, добавление новых таблиц для хранения дополнительной информации, а также внедрение автоматизации процессов с использованием триггеров и хранимых процедур.

В результате данной работы была создана функциональная база данных, способная эффективно управлять данными о клиентах и расчетах, а также предоставлять ценную аналитическую информацию. Эта подсистема расчетов с клиентами позволит телекоммуникационной компании улучшить свои процессы и принимать обоснованные управленческие решения на основе достоверной информации.

## СПИСОК ИСПОЛЬЗОВАННЫХ ЛИТЕРАТУР

1. <https://www.jetbrains.com/help/datagrip/installation-guide.html>
2. <https://www.jetbrains.com/ru-ru/datagrip/features/navigation.html>
3. <https://www.jetbrains.com/ru-ru/datagrip/features/mysql.html>
4. <https://proglib.io/p/sqlite-tutorial>
5. <https://clck.ru/34a4qX>
6. <https://nuancesprog.ru/p/10647/>
7. Рассуждение на тему, какую базу данных выбирать. URL: <https://habr.com/ru/post/348220/>
8. Календарные типы данных в MySQL: особенности использования. URL: <https://habr.com/ru/post/69983/>
9. Основы работы с MySQL Workbench: быстрый старт, управление схемой данных. URL: <https://mithrandir.ru/professional/soft-and-hardware/mysql-workbench-basics.html>
10. \_Макин, Дж.К. Проектирование серверной инфраструктуры баз данных Microsoft SQL Server 2005 / Дж.К. Макин. - М.: Русская редакция, 2008
- 11.Малыхина, М.П. Базы данных: основы, проектирование, использование / М.П. Малыхина. - СПб.: BHV, 2007
- 12.Мартишин, С.А. Проектирование и реализация баз данных в СУБД MySQL с использованием MySQL Workbench: Методы и средства проектирования информационных систем и техноло / С.А. Мартишин, В.Л. Симонов, М.В. Храпченко. - М.: Форум, 2018